Product Requirements Document (PRD)

Executive Summary

Purpose: Define the comprehensive product requirements for Habit Planner — an AI-powered personal growth platform that empowers users to build sustainable habits, track progress, and achieve meaningful life improvements through gamification, intelligent insights, and beautiful user experiences.

Project Overview

- **Project Name**: Habit Planner (Personal_world)
- **Project Type**: Web App (Full-Stack SaaS Platform)
- **Brief Description**: A premium, portfolio-quality habit tracking and personal productivity platform built with Next.js and Supabase, combining the best UX patterns from Notion, Linear, TickTick, Todoist, Duolingo, and Apple Health.
- **Owner**: Vanga Rudiveek
- **Date**: 2026-07-31

Problem Statement

Business Problem: Millions of people struggle to build and maintain positive habits due to lack of structure, accountability, and motivation. Existing solutions are either too simplistic (basic to-do lists) or too complex (enterprise project management tools), creating a gap for an intelligently designed personal growth platform.

- **Current Pain Point**: Users abandon habit-building apps within 2 weeks due to lack of engagement, poor visualization of progress, and absence of meaningful feedback loops.
- **Impact**: 80% of New Year's resolutions fail by February; users cycle through multiple apps without achieving lasting behavior change, leading to frustration and diminished self-efficacy.
- **Root Cause**: Current tools lack the combination of beautiful UX, gamification psychology, data-driven insights, and AI-powered personalization needed to sustain long-term engagement.

Proposed Solution

Solution Overview: Habit Planner delivers a premium, all-in-one personal growth platform with habit tracking, goal management, analytics, gamification, smart reminders, and future AI coaching — all wrapped in an elegant, responsive interface that makes habit building enjoyable and sustainable.

- **Approach**: Build a modular Next.js application with Supabase backend, implementing progressive engagement through gamification (XP, streaks, badges), rich analytics (heatmaps, trends), and smart scheduling — with a clear roadmap to AI-powered features.

  Key Differentiators:
  - Premium UX inspired by world-class apps (Notion + Linear + Duolingo design language)
  - Comprehensive gamification system (XP, levels, badges, streaks, milestones) driving sustained engagement
  - Rich analytics with heatmaps, trend analysis, and actionable insights
  - Modular architecture designed for future AI integration (habit prediction, personalized coaching)

Expected Business Benefits

- **Primary Benefit**: Create a portfolio-quality, production-ready SaaS application demonstrating full-stack engineering excellence while solving a genuine user need.

  Secondary Benefits:

    - Achieve 40%+ 30-day user retention through gamification and smart engagement
    - Generate measurable habit completion improvements (target: 65%+ daily completion rate among active users)
    - Establish a scalable platform architecture supporting 10,000+ concurrent users
- **ROI Estimate**: Platform positioned as a portfolio flagship demonstrating modern full-stack architecture, scalable design patterns, and AI-readiness — with potential to convert to commercial SaaS generating $5K-$50K MRR at scale.
- **Risk of Not Doing**: Missing the opportunity to establish a production-grade portfolio piece in the growing $10B+ personal productivity market; losing competitive advantage as AI-first habit apps emerge.

Key Success Metrics

- **Metric 1**: 30-day user retention rate ≥ 40% (industry average: 25%)
- **Metric 2**: Daily habit completion rate ≥ 65% among active users
- **Metric 3**: Application Lighthouse performance score ≥ 90 across all metrics
- **Metric 4**: User engagement: average session duration ≥ 4 minutes, ≥ 3 sessions/week

---

1. Background & Context

1.1 Current Situation

Description: The personal productivity and habit tracking market is saturated with apps ranging from bare-bones trackers to enterprise-grade project management tools. Users seeking meaningful personal growth tools face a fragmented landscape where no single solution combines elegant design, behavioral psychology, gamification, comprehensive analytics, and AI-powered insights.

- **Existing Solutions**: Habitica (gamification-heavy, cluttered UI), Streaks (iOS-only, limited), TickTick (task-focused, habits are secondary), Notion (requires extensive setup), Loop Habit Tracker (Android-only, basic)
- **Their Limitations**: No single solution combines premium UX + comprehensive gamification + rich analytics + calendar integration + AI readiness. Most are platform-locked or require significant manual configuration.
- **Previous Attempts**: Traditional to-do list apps added "habits" as an afterthought; gamification-first apps (Habitica) sacrifice UX elegance; health apps (Apple Health) lack habit-building methodology.

1.2 Market & Competitive Context

- **Market Drivers**: Growing focus on mental health and well-being post-pandemic; rise of "quantified self" movement; increasing acceptance of gamification in personal tools; AI democratization enabling personalized coaching.
- **Competitive Landscape**: Habitica ($5M+ revenue, gamification leader), Fabulous (behavior science approach, $30M raised), Streaks (Apple Design Award winner, iOS-only), TickTick (cross-platform, habits as secondary feature).
- **Industry Benchmarks**: Top habit apps achieve 25-35% 30-day retention; average session duration 2-5 minutes; premium conversion rates of 3-8%.

1.3 Strategic Alignment

- **Company Strategy**: Build a portfolio-quality full-stack application demonstrating mastery of modern web technologies (Next.js, Supabase, Tailwind CSS) while creating genuine user value.
- **Strategic Initiatives**: AI-powered personal productivity tools; premium web application development; scalable SaaS architecture patterns.
- **Long-term Vision**: Evolve into a comprehensive AI-powered personal growth ecosystem with habit coaching, wellness tracking, and intelligent life optimization — potentially commercializable as a SaaS product.

1.4 Regulatory & Compliance Context

- **Regulatory Requirements**: GDPR compliance for EU users (data deletion, export rights); CCPA compliance for California users.
- **Compliance Standards**: SOC 2 Type I alignment for data security practices; OWASP Top 10 security compliance.
- **Audit Requirements**: User data export capability; activity logging for security events; transparent privacy policy.

---

2. Business Objectives & Success Criteria

2.1 Primary Business Objective

Objective: Deliver a production-ready, portfolio-quality habit tracking platform with 10+ core modules within 16 weeks that achieves a Lighthouse performance score ≥ 90, supports 1,000+ concurrent users, and demonstrates full-stack engineering excellence across frontend, backend, database design, and UX.

2.2 Secondary Objectives

Objective 1: Achieve 40%+ 30-day user retention through gamification, streaks, and engagement loops

- **Rationale**: Retention validates that the app provides genuine ongoing value, not just novelty

Objective 2: Build a modular, extensible architecture that supports AI feature integration within 4 weeks of core platform launch

- **Rationale**: Future-proofing ensures the platform can evolve without major rewrites

Objective 3: Deliver WCAG 2.1 AA compliant accessibility across all modules with dark/light mode support

- **Rationale**: Inclusive design expands the addressable user base and demonstrates professional quality

2.3 Success Metrics & KPIs

Metric	Baseline	Target	Timeline	Measurement Method
User Retention (30-day)	0% (new product)	≥ 40%	3 months post-launch	Supabase analytics: returning users / total signups
Daily Habit Completion Rate	N/A	≥ 65%	1 month post-launch	Completed habits / assigned habits per day
Lighthouse Performance Score	N/A	≥ 90 (all categories)	At launch	Google Lighthouse audit
Average Session Duration	N/A	≥ 4 minutes	2 months post-launch	Client-side analytics tracking

Time to Interactive (TTI)	N/A	< 2.5 seconds	At launch	Lighthouse / Web Vitals
Streak Maintenance Rate	N/A	≥ 50% users with 7+ day streak	2 months post-launch	Database query: users with active streak ≥ 7
Feature Adoption (Gamification)	N/A	≥ 70% users earn first badge	1 month post-launch	Badge award events / total active users
Accessibility Compliance	N/A	WCAG 2.1 AA pass	At launch	axe-core automated testing + manual audit

2.4 Project Priority

- **Overall Priority**: Critical
- **Business Urgency**: Portfolio and market timing — AI-powered productivity tools are experiencing rapid growth; early market presence establishes credibility and user base.
- **Rationale**: The combination of portfolio value, market opportunity, and technical learning objectives makes this a critical-priority project requiring focused execution.

---

3. Stakeholder Analysis

3.1 Stakeholder Matrix

Stakeholder Group	Key Representatives	Role	Interest/Concern	Key Requirements	Impact Level
End Users	Students, Professionals, Health-conscious users	Primary users	Easy habit tracking, motivation, insights	Intuitive UX, gamification, analytics	Critical
Product Owner	Vanga Rudiveek	Decision maker	Portfolio quality, technical excellence	Production-ready, scalable, AI-ready	Critical
Development Team	Full-stack developers	Implementers	Clean architecture, maintainability	Modular code, clear specs, testing	High

Design Stakeholders	UX/UI review	Quality assurance	Premium aesthetics, accessibility	WCAG compliance, responsive design	High
Future Users (AI)	Power users	Early adopters	AI coaching, predictions	AI integration readiness	Medium
Open Source Community	Contributors	Collaborators	Code quality, documentation	Well-documented, extensible	Medium

3.2 User Personas

Persona 1: Priya — The Ambitious Student

- **Description**: 21-year-old computer science student balancing academics, personal development, and fitness goals
- **Demographics**: Age 18-25, urban, high technical proficiency, uses multiple productivity tools
  Goals:
  - Build consistent study habits (2 hours focused study daily)
  - Track fitness routine (gym 4x/week, meditation daily)
  - Maintain work-life balance through structured routines

  Pain Points:
  - Forgets habits when schedule changes
  - Loses motivation after breaking a streak
  - Can't visualize long-term progress
  - Current apps feel like chores, not rewards

  Needs:
  - Quick daily check-in (< 30 seconds to mark habits complete)
  - Gamification that makes tracking feel rewarding
  - Visual progress that shows improvement over weeks/months
  - Smart reminders that adapt to her schedule
- **Behaviors**: Checks phone 50+ times/day; responds well to streaks and leaderboards; uses dark mode exclusively; prefers mobile-first but also uses laptop
- **Success Criteria**: Maintains 80%+ completion rate for 30+ days; earns Level 10 in first month

Persona 2: Marcus — The Productivity-Focused Professional

- **Description**: 34-year-old product manager at a tech company seeking to optimize daily routines and track personal KPIs
- **Demographics**: Age 28-40, suburban, high technical proficiency, uses Notion/Linear/Todoist daily
  Goals:
  - Track 10+ daily habits across work and personal life
  - Analyze productivity patterns to optimize routines
  - Set and achieve quarterly personal goals

  Pain Points:
  - Existing tools don't provide meaningful analytics
  - Too many apps; wants a single source of truth

    - Needs flexibility in habit scheduling (weekdays vs weekends)
    - Wants data export for personal analysis

    Needs:
    - Rich analytics dashboard with trends and patterns
    - Flexible scheduling (custom repeat patterns)
    - Goal setting with measurable milestones
    - Data export in standard formats (CSV, JSON)
    - Clean, distraction-free UI similar to Linear
- **Behaviors**: Power user; customizes everything; reviews weekly analytics; exports data for personal dashboards; prefers keyboard shortcuts
- **Success Criteria**: Uses analytics weekly; maintains 15+ habits; achieves 3/4 quarterly goals

Persona 3: Sarah — The Health-Conscious Wellness Seeker

- **Description**: 28-year-old yoga instructor building holistic wellness routines encompassing fitness, nutrition, mindfulness, and sleep
- **Demographics**: Age 25-35, urban, moderate technical proficiency, values aesthetics and simplicity

    Goals:
    - Track wellness habits: meditation, hydration, sleep, exercise
    - Build progressive difficulty (10 min $\rightarrow$ 30 min meditation)
    - Connect habits to broader wellness goals

    Pain Points:
    - Health apps are clinical and cold; wants warmth and beauty
    - Overwhelmed by complex interfaces
    - Needs gentle accountability, not aggressive notifications
    - Wants to see mind-body connections in her data

    Needs:
    - Beautiful, calming interface with smooth animations
    - Simple habit creation with progressive difficulty
    - Gentle, encouraging reminder system
    - Wellness-focused categories and achievements
    - Light mode with soft, natural colors
- **Behaviors**: Checks app morning and evening; appreciates motivational quotes; shares achievements on social media; values streaks but needs compassionate streak recovery
- **Success Criteria**: Daily engagement for 60+ days; tracks 5+ wellness habits consistently; feels motivated (not guilty) when using the app

Persona 4: Raj — The Lifelong Learner

- **Description**: 45-year-old entrepreneur building knowledge habits including reading, language learning, skill development, and journaling
- **Demographics**: Age 35-55, global, moderate-high technical proficiency, values consistency over intensity

    Goals:
    - Read 30 minutes daily; learn Spanish 15 min/day
    - Journal every evening; practice public speaking weekly
    - Track learning streaks and knowledge milestones

    Pain Points:
    - Forgets habits that aren't daily (weekly, biweekly)
    - Needs habit notes to track what was learned
    - Wants to see cumulative time invested in learning

- Struggles with habit restart after travel/vacation

  Needs:
  - Flexible repeat schedules (daily, weekly, custom)
  - Habit notes for journaling/reflection
  - Cumulative progress tracking (total hours read)
  - Vacation mode / streak freeze
  - Calendar view showing habit history
- **Behaviors**: Methodical planner; reviews monthly progress; values data and insights; prefers desktop for journaling; uses mobile for quick check-ins
- **Success Criteria**: Maintains learning habits for 90+ days; accumulates 100+ hours in tracked activities; achieves monthly learning goals

---

4. User Stories & Use Cases

4.1 User Stories (Extensive List)

> **Format**: As a [actor], I want [feature/capability], so that [benefit/value]

Core User Stories (Must Have — P0):

1. As a new user, I want to register with email and password, so that I can create my personal account securely
2. As a returning user, I want to log in with my credentials, so that I can access my habits and progress
3. As a user who forgot my password, I want to reset it via email, so that I can regain account access
4. As a logged-in user, I want to see my dashboard with today's habits and progress, so that I know what to focus on
5. As a user, I want to create a new habit with name, category, priority, and schedule, so that I can start tracking it
6. As a user, I want to mark habits as complete for today, so that I can track my daily progress
7. As a user, I want to view my habit streak count, so that I feel motivated to maintain consistency
8. As a user, I want to see my productivity score on the dashboard, so that I can gauge my overall performance
9. As a user, I want to edit existing habits (name, schedule, category), so that I can adapt them as my needs change
10. As a user, I want to delete or archive habits I no longer need, so that my active list stays relevant
11. As a user, I want to view a calendar showing my habit completion history, so that I can see patterns over time
12. As a user, I want to see a heatmap of my activity, so that I can visualize consistency at a glance
13. As a user, I want to set reminders for specific habits, so that I don't forget important routines
14. As a user, I want to view analytics showing my completion rates over time, so that I can track improvement
15. As a user, I want to earn XP when I complete habits, so that I feel rewarded for consistency

Secondary User Stories (Should Have — P1): 16. As a user, I want to set daily, weekly, and monthly goals, so that I have clear targets to achieve 17. As a user, I want to earn badges for reaching milestones, so that I feel a sense of accomplishment 18. As a user, I want to level up based on XP earned, so that I can see my growth over time 19. As a user, I want to add notes to habit completions, so that I can reflect on my experience 20. As a user, I want to categorize habits (Health, Productivity, Learning, etc.), so that I can organize my tracking 21. As a user, I want to set habit difficulty levels, so that harder habits award more XP 22. As a user, I want to see weekly and monthly overview charts, so that I can identify trends 23. As a user, I want to receive a motivational quote on my dashboard, so that I start my day inspired 24. As a user, I want to export my habit data as CSV/JSON, so that I can analyze it externally 25. As a user, I want to switch between dark mode and light mode, so that I can use the app comfortably in any environment 26. As a user, I want to see upcoming reminders on my dashboard, so that I can prepare for scheduled activities 27. As a user, I want to track habit completion percentage per category, so that I can identify weak areas 28. As a user, I want to see my longest streak for each habit, so that I know my personal records 29. As a user, I want to view habit trends (improving, declining, stable), so that I can adjust my approach 30. As a user, I want to set custom repeat schedules (e.g., Mon/Wed/Fri), so that habits fit my real routine

Enhancement User Stories (Could Have — P2): 31. As a user, I want a streak freeze option, so that I don't lose progress during sick days or vacations 32. As a user, I want to share achievements on social media, so that I can celebrate milestones with friends 33. As a user, I want keyboard shortcuts for common actions, so that I can be more efficient 34. As a user, I want to import habits from other apps, so that I can migrate without starting over 35. As a user, I want to set habit templates for common routines, so that new users can get started quickly 36. As a user, I want progressive habit difficulty (auto-increase), so that I'm always challenged appropriately 37. As a user, I want to group habits into routines (morning, evening), so that I can batch-complete related habits 38. As a user, I want cumulative time tracking for habits, so that I know total hours invested 39. As a user, I want to view my achievements on a dedicated page, so that I can review my accomplishments 40. As a user, I want the app to suggest optimal habit scheduling based on my patterns, so that I can maximize completion rates

Future AI User Stories (Won't Have This Release): 41. As a user, I want AI-powered habit suggestions based on my goals, so that I can discover effective habits 42. As a user, I want an AI daily planning assistant, so that I can optimize my schedule 43. As a user, I want AI-generated motivation messages personalized to my journey, so that I stay engaged 44. As a user, I want AI to predict which habits I'm at risk of breaking, so that I can take preventive action 45. As a user, I want an AI coach that adapts recommendations based on my performance, so that I continuously improve

4.2 Key User Journeys

Journey 1: New User Onboarding & First Habit

- **Actor**: New user (any persona)
- **Trigger**: User discovers Habit Planner and wants to start tracking habits
- **Preconditions**: User has a valid email address; application is accessible

  Steps:

  1. **Visit Landing/Register Page**: User sees the app value proposition → System displays registration form
  2. **Complete Registration**: User enters email, password, confirms password → System validates input, creates account, sends verification
  3. **Email Verification**: User clicks verification link → System activates account
  4. **First Login**: User logs in with credentials → System authenticates and redirects to dashboard
  5. **Empty Dashboard State**: User sees onboarding prompts → System shows "Create your first habit" CTA
  6. **Create First Habit**: User clicks CTA, fills habit form (name, category, schedule, priority) → System saves habit, awards "First Habit" badge (+50 XP)
  7. **Dashboard Update**: User sees new habit on dashboard → System displays habit card with completion toggle
  8. **Complete First Habit**: User marks habit as done → System awards XP, updates streak to 1, shows congratulation micro-animation
- **Expected Outcome**: User has an active account with one habit, has experienced the reward loop (XP + badge), and understands the core interaction pattern
- **Success Criteria**: Time from registration to first habit completion < 3 minutes; user receives positive feedback (badge, XP animation)

  Alternative Paths:

  - **Alt Path 1**: User registers via OAuth (Google) — skips email verification step
  - **Alt Path 2**: User browses templates first, then selects a pre-built habit

  Error Scenarios:

  - **Error 1**: Email already registered → System shows "Account exists" message with login link
  - **Error 2**: Weak password → System shows specific requirements (8+ chars, 1 uppercase, 1 number)
  - **Error 3**: Network error during registration → System preserves form state, shows retry option

Journey 2: Daily Habit Check-in

- **Actor**: Returning user (Priya — The Ambitious Student)

- **Trigger**: User opens app in the morning to review and complete today's habits
- **Preconditions**: User is logged in; has 5+ active habits configured

  Steps:

  1. **Open Dashboard**: User navigates to app → System loads personalized dashboard with today's habits, streak info, motivational quote
  2. **Review Today's Habits**: User scans habit list → System shows each habit with category color, priority indicator, and completion status
  3. **Complete Habits Sequentially**: User taps/clicks completion toggle for each finished habit → System animates completion, awards XP per habit, updates productivity score in real-time
  4. **Add Note (optional)**: User adds reflection note to meditation habit → System saves note with timestamp
  5. **View Progress**: User checks productivity score (now 80%) → System shows daily progress bar and streak counter
  6. **Receive Streak Milestone**: User hits 7-day streak → System triggers milestone celebration animation, awards "Week Warrior" badge
- **Expected Outcome**: User completes daily check-in in < 60 seconds, feels motivated by XP gains and streak maintenance
- **Success Criteria**: Daily check-in time < 90 seconds for 5 habits; productivity score updates in real-time; streak celebration triggers correctly

  Alternative Paths:

  - **Alt Path 1**: User partially completes habits — productivity score reflects partial progress
  - **Alt Path 2**: User misses a day — system shows streak status, offers encouragement (not guilt)

  Error Scenarios:

  - **Error 1**: Offline mode — completions queue locally, sync when connection restores
  - **Error 2**: Accidental completion — user can undo within 5 seconds via toast notification

Journey 3: Weekly Analytics Review

- **Actor**: Marcus — The Productivity-Focused Professional
- **Trigger**: End of week; user wants to analyze performance and plan adjustments
- **Preconditions**: User has 7+ days of tracking data; has multiple habit categories

  Steps:

  1. **Navigate to Analytics**: User clicks Analytics tab → System loads analytics dashboard with weekly overview as default
  2. **Review Completion Rates**: User sees weekly completion percentage per category → System displays bar chart and percentage cards
  3. **Identify Trends**: User examines trend indicators → System shows improvement/decline arrows per habit with percentage change
  4. **Deep Dive into Category**: User clicks "Health" category → System shows detailed breakdown of health habits with individual completion rates
  5. **View Heatmap**: User switches to heatmap view → System renders GitHub-style contribution heatmap showing daily activity intensity
  6. **Export Data**: User clicks export → System generates CSV with date, habit, completion status, notes, XP earned
  7. **Set Improvement Goal**: User navigates to Goals, creates "Improve Health habits to 80%" → System saves goal, links to relevant habits
- **Expected Outcome**: User gains actionable insights into performance patterns, identifies weak areas, and sets specific improvement goals
- **Success Criteria**: Analytics page loads in < 2 seconds; charts render correctly with 7+ days of data; export completes in < 5 seconds

  Alternative Paths:

  - **Alt Path 1**: User with < 7 days data sees "Keep tracking!" message with progress toward first analytics unlock

- **Alt Path 2**: User switches time range (daily/monthly/yearly) — charts re-render with appropriate aggregation

Error Scenarios:

- **Error 1**: No data for selected period → System shows empty state with guidance to track more
- **Error 2**: Chart rendering failure → System shows data in table format as fallback

Journey 4: Achievement Unlocked & Level Up

- **Actor**: Sarah — The Health-Conscious Wellness Seeker
- **Trigger**: User accumulates enough XP to level up and unlock a new badge
- **Preconditions**: User is near XP threshold for next level; has been consistently tracking

  Steps:

  1. **Complete Habit**: User marks morning meditation as complete → System awards XP (30 XP for medium difficulty)
  2. **XP Threshold Reached**: Total XP crosses level boundary → System triggers level-up animation
  3. **Level Up Celebration**: Full-screen celebration with confetti, new level badge, unlocked perks → User sees "Level 5 — Habit Champion" title
  4. **Badge Award**: Simultaneous badge unlock ("30-Day Meditator") → System shows badge details and rarity
  5. **Achievement Gallery Update**: User views achievements page → System shows all earned badges, current level, XP progress bar to next level
  6. **Motivation Boost**: User sees progress reinforcement → System displays "You're in the top 20% of users!" (if applicable)
- **Expected Outcome**: User experiences dopamine reward from level-up and badge; feels motivated to continue; engagement deepens
- **Success Criteria**: Level-up animation plays smoothly (60fps); badge appears in gallery within 1 second; XP calculation is accurate

  Alternative Paths:

  - **Alt Path 1**: User earns multiple badges simultaneously — system queues celebrations, showing one at a time
  - **Alt Path 2**: User dismisses celebration — can review in achievements page later

  Error Scenarios:

  - **Error 1**: XP calculation error — system recalculates from source of truth (completion records)
  - **Error 2**: Animation performance issue on low-end device — graceful degradation to simple notification

4.3 Use Cases (Detailed Scenarios)

UC-01: Habit Creation

- **Actor**: Authenticated user
- **Goal**: Create a new habit with full configuration
- **Preconditions**: User is logged in; on habits page or dashboard

  Main Flow:

  1. User clicks "Create Habit" button
  2. System displays habit creation form/modal
  3. User enters habit name (required, 1-100 characters)
  4. User selects category from dropdown (Health, Productivity, Learning, Wellness, Custom)
  5. User sets priority (Low, Medium, High, Critical)
  6. User sets difficulty (Easy: 10 XP, Medium: 20 XP, Hard: 30 XP, Expert: 50 XP)
  7. User configures repeat schedule (Daily, Weekdays, Weekends, Custom days)
  8. User optionally sets time-of-day preference (Morning, Afternoon, Evening, Anytime)
  9. User clicks "Create Habit"
  10. System validates all inputs, saves habit to database

11. System displays success toast with habit name
12. Habit appears in user's habit list and on relevant calendar days

- **Postconditions**: Habit exists in database; appears on dashboard for scheduled days; is included in analytics calculations

  Alternative Flows:
  - **Alt 1**: User cancels creation — form closes, no data saved
  - **Alt 2**: User creates from template — form pre-fills with template values
  - **Alt 3**: User creates from dashboard quick-add — simplified form with only name and category

  Exception Flows:
  - **Exc 1**: Duplicate habit name — system warns "You already have a habit named X. Continue anyway?"
  - **Exc 2**: Network error on save — system retries 3x, then shows error with "Save Draft" option
  - **Exc 3**: Maximum habit limit reached (50 active) — system suggests archiving inactive habits

UC-02: Daily Habit Completion

- **Actor**: Authenticated user
- **Goal**: Mark one or more habits as complete for the current day
- **Preconditions**: User is logged in; has habits scheduled for today

  Main Flow:
  1. User views dashboard or habits page
  2. System displays today's scheduled habits with completion toggles
  3. User clicks completion toggle for a habit
  4. System immediately updates UI (optimistic update)
  5. System saves completion record (habit_id, user_id, date, timestamp)
  6. System calculates and awards XP based on habit difficulty
  7. System updates streak counter for that habit
  8. System recalculates daily productivity score
  9. System checks for badge/milestone triggers
  10. If milestone reached, system triggers celebration UI
- **Postconditions**: Completion recorded in database; XP awarded; streak updated; analytics data updated; productivity score recalculated

  Alternative Flows:
  - **Alt 1**: User uncompletes a habit (undo) — XP deducted, streak recalculated, analytics updated
  - **Alt 2**: User completes all habits — "Perfect Day" celebration triggered, bonus XP awarded
  - **Alt 3**: User completes habit with note — note saved alongside completion record

  Exception Flows:
  - **Exc 1**: Completion fails to save — toast error shown, completion toggle reverts, retry available
  - **Exc 2**: User tries to complete future-dated habit — system blocks with "Not yet! Check back tomorrow"

UC-03: Analytics Dashboard Interaction

- **Actor**: Authenticated user with 7+ days of data
- **Goal**: Review performance analytics and identify trends
- **Preconditions**: User has sufficient tracking history; is on analytics page

  Main Flow:
  1. User navigates to Analytics section
  2. System loads default view (weekly overview)
  3. System renders: completion rate card, streak card, XP card, category breakdown chart
  4. User selects time range (daily/weekly/monthly/yearly)
  5. System re-fetches and re-renders charts for selected range

6. User clicks specific category to drill down
7. System shows per-habit breakdown within category
8. User views heatmap for activity visualization
9. System renders 365-day heatmap with color intensity based on completion count
10. User exports data for selected range
11. System generates and downloads CSV/JSON file

- **Postconditions**: User has viewed analytics; any exports are saved to device; no data modified
  Alternative Flows:
  - **Alt 1**: Insufficient data (< 7 days) — show "Keep going!" with progress to unlock analytics
  - **Alt 2**: User filters by specific habit — all charts recalculate for single habit
  Exception Flows:
  - **Exc 1**: Chart library fails to render — fallback to data table view
  - **Exc 2**: Export timeout for large datasets — stream download with progress indicator

---

5. Scope Definition

5.1 In-Scope Features & Capabilities

Category 1: Authentication & User Management

[Feature 1.1]: User Registration

- **Description**: Email/password registration with form validation, email verification, and secure account creation
- **User Value**: Users can create personal accounts to track habits privately and securely
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Email + password registration form with real-time validation
  - Password strength indicator (min 8 chars, 1 uppercase, 1 number, 1 special)
  - Email verification flow with resend capability
  - Duplicate email detection with helpful messaging

[Feature 1.2]: User Authentication (Login/Logout)

- **Description**: Secure login with JWT tokens, session management, and protected route enforcement
- **User Value**: Users can securely access their personal data across sessions
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Email/password login with "Remember Me" option
  - JWT token generation and refresh mechanism
  - Automatic session expiry after inactivity (configurable, default 7 days)
  - Protected route middleware redirecting unauthenticated users
  - Secure logout clearing all session data

[Feature 1.3]: Password Recovery

- **Description**: Forgot password flow with secure email-based reset link
- **User Value**: Users can recover access without losing their data
- **Priority**: Must Have (P0)
  Key Capabilities:

- "Forgot Password" link on login page
- Secure reset token generation (expires in 1 hour)
- Email with reset link sent within 30 seconds
- New password form with strength validation
- Confirmation of successful reset

[Feature 1.4]: Profile Management

- **Description**: User profile page for managing personal information, avatar, and preferences
- **User Value**: Users can personalize their account and manage their identity
- **Priority**: Should Have (P1)
  Key Capabilities:
  - Display name, avatar upload/selection
  - Email change with re-verification
  - Password change with current password confirmation
  - Account deletion with data export option
  - Timezone and locale settings

Category 2: Dashboard

[Feature 2.1]: Main Dashboard

- **Description**: Centralized overview page showing today's habits, progress, score, and key metrics at a glance
- **User Value**: Users get immediate context on their day's priorities and progress without navigating to multiple pages
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Welcome section with personalized greeting (time-of-day aware)
  - Daily productivity score (0-100%) calculated from habit completions
  - Today's habits list with completion toggles
  - Current streak display (longest active streak)
  - Quick stats cards (total XP, level, habits completed today)
  - Motivational quote (rotated daily)
  - Calendar preview (mini month view with completion indicators)
  - Upcoming reminders list (next 3-5 reminders)
  - Weekly overview chart (7-day completion bar chart)

Category 3: Habits Module

[Feature 3.1]: Habit CRUD Operations

- **Description**: Full create, read, update, delete functionality for habits with rich configuration options
- **User Value**: Users can fully manage their habit library to match their evolving goals
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Create habit: name, description, category, priority, difficulty, schedule, color, icon
  - Edit habit: modify any field, effective immediately
  - Delete habit: soft delete with 30-day recovery option
  - Archive habit: remove from active list without deleting history
  - Habit list view with sorting (name, priority, category, streak) and filtering
  - Habit detail view showing full history and stats

[Feature 3.2]: Habit Tracking & Completion

- **Description**: Daily habit completion system with optimistic UI updates, XP awards, and streak management
- **User Value**: Users can quickly log completions and immediately see the impact on their progress
- **Priority**: Must Have (P0)
  Key Capabilities:
  - One-click/tap completion toggle
  - Optimistic UI update (instant feedback before server confirmation)
  - XP award calculation based on difficulty
  - Streak increment and tracking (current + longest)
  - Undo completion within 5-second window
  - Habit notes attached to specific completions
  - Batch completion for habit groups/routines

[Feature 3.3]: Habit Scheduling

- **Description**: Flexible repeat schedule configuration supporting various patterns
- **User Value**: Users can configure habits to match their real-life schedule precisely
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Daily (every day)
  - Weekdays only (Mon-Fri)
  - Weekends only (Sat-Sun)
  - Custom days (any combination of days)
  - Specific times of day (Morning, Afternoon, Evening) for ordering
  - Start date and optional end date

Category 4: Calendar Module

[Feature 4.1]: Calendar Views

- **Description**: Multi-view calendar showing habit history and scheduled activities
- **User Value**: Users can visualize their habit patterns temporally and plan ahead
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Daily view: full list of habits for selected day with completion status
  - Weekly view: 7-day grid showing habits per day with completion indicators
  - Monthly view: traditional calendar with color-coded completion intensity per day
  - Navigation: previous/next period, jump to today, date picker

[Feature 4.2]: Activity Heatmap

- **Description**: GitHub-style contribution heatmap showing habit activity intensity over time
- **User Value**: Users can instantly see consistency patterns and identify gaps in their routine
- **Priority**: Should Have (P1)
  Key Capabilities:
  - 365-day heatmap with 5 intensity levels (0, 1-25%, 26-50%, 51-75%, 76-100%)
  - Color coding per category (optional toggle)
  - Tooltip showing exact count and date on hover
  - Responsive layout adapting to screen width

Category 5: Analytics Module

[Feature 5.1]: Performance Analytics

- **Description**: Comprehensive analytics dashboard with charts, trends, and insights
- **User Value**: Users can understand their behavior patterns and make data-driven improvements
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Overall completion rate (daily/weekly/monthly/yearly)
  - Per-category completion breakdown (bar chart)
  - Trend analysis (improving/declining/stable indicators per habit)
  - Time-of-day completion patterns
  - Best performing and worst performing habits

[Feature 5.2]: Visualizations

- **Description**: Rich chart library providing multiple visualization types for habit data
- **User Value**: Users can consume complex data through intuitive visual formats
- **Priority**: Should Have (P1)
  Key Capabilities:
  - Line charts: completion rate over time
  - Bar charts: category comparison, daily distribution
  - Pie/Donut charts: category breakdown, time-of-day distribution
  - Heatmaps: yearly activity overview
  - Progress radials: goal completion percentage

[Feature 5.3]: Data Export

- **Description**: Export habit data in standard formats for external analysis
- **User Value**: Power users can analyze data in spreadsheets or custom tools
- **Priority**: Should Have (P1)
  Key Capabilities:
  - CSV export with all fields (date, habit, category, completion, XP, notes)
  - JSON export for programmatic use
  - Date range selection for export
  - Per-habit or all-habits export options

Category 6: Reminders Module

[Feature 6.1]: Reminder System

- **Description**: Configurable reminder system alerting users about upcoming and due habits
- **User Value**: Users don't forget habits, especially those scheduled at specific times
- **Priority**: Should Have (P1)
  Key Capabilities:
  - Per-habit reminder configuration (time, frequency)
  - Dashboard reminder cards showing upcoming items
  - Browser notification support (with user permission)
  - Daily summary reminder (configurable time)
  - Smart reminders based on typical completion time patterns

Category 7: Achievements & Gamification

[Feature 7.1]: XP & Leveling System

- **Description**: Experience point system that rewards habit completion with cumulative XP driving level progression
- **User Value**: Users feel tangible progress and achievement through a clear leveling system
- **Priority**: Must Have (P0)
  Key Capabilities:
  - XP earned per habit completion based on difficulty (Easy: 10, Medium: 20, Hard: 30, Expert: 50)
  - Bonus XP for streaks (7-day: +50, 30-day: +200, 100-day: +500)
  - Bonus XP for "Perfect Day" (all habits completed: +25)
  - Level progression: Level = floor(sqrt(totalXP / 100))
  - Level-up celebration animation
  - XP progress bar showing distance to next level

[Feature 7.2]: Badge System

- **Description**: Achievement badges awarded for reaching specific milestones and accomplishments
- **User Value**: Users collect meaningful achievements that recognize their dedication and progress
- **Priority**: Should Have (P1)
  Key Capabilities:
  - Milestone badges: "First Habit", "7-Day Streak", "30-Day Streak", "100-Day Streak"
  - Category mastery badges: "Health Guru", "Productivity Pro", "Learning Legend"
  - Volume badges: "100 Completions", "500 Completions", "1000 Completions"
  - Special badges: "Perfect Week", "Perfect Month", "Early Bird" (all morning habits before 8am)
  - Badge gallery page with earned/locked badge grid
  - Badge rarity indicators (Common, Rare, Epic, Legendary)

[Feature 7.3]: Streaks & Milestones

- **Description**: Streak tracking system with milestone recognition and streak protection
- **User Value**: Users are motivated to maintain consistency through visible streak progress and milestone celebrations
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Per-habit current streak and all-time longest streak
  - Global streak (days with at least one habit completed)
  - Streak milestone celebrations (7, 14, 30, 60, 90, 100, 365 days)
  - Streak freeze / vacation mode (P2 enhancement)
  - Visual streak indicators on habit cards

Category 8: Goals Module

[Feature 8.1]: Goal Management

- **Description**: Goal setting and tracking system for daily, weekly, monthly, and custom time-frame goals
- **User Value**: Users can set clear targets and track progress toward meaningful outcomes
- **Priority**: Should Have (P1)
  Key Capabilities:
  - Create goals with title, description, target metric, deadline
  - Goal types: daily (e.g., "Complete 5 habits"), weekly (e.g., "80% completion"), monthly, custom
  - Progress tracking with visual progress bar
  - Goal-habit linking (which habits contribute to which goals)

- Goal status: Active, Achieved, Failed, Paused
- Goal history and achievement rate

Category 9: Settings Module

[Feature 9.1]: Application Settings

- **Description**: Comprehensive settings page for customizing app behavior and appearance
- **User Value**: Users can tailor the application to their preferences and needs
- **Priority**: Must Have (P0)
  Key Capabilities:
  - Theme toggle: Dark Mode / Light Mode / System default
  - Notification preferences: enable/disable, quiet hours, channels
  - Language selection (English primary; i18n-ready architecture)
  - Timezone configuration
  - Data management: export all data, import data, delete account
  - Privacy settings: data sharing preferences

5.2 Out-of-Scope

Excluded from This Release:

Native Mobile Applications (iOS/Android)

- **Reason**: Focused on web-first with responsive design; native apps require separate codebase and significant additional effort
- **Future Plans**: Phase 2 — React Native or Flutter mobile app sharing API backend

Social Features (Friends, Leaderboards, Groups)

- **Reason**: Adds complexity to data model and privacy considerations; core value is personal tracking
- **Future Plans**: Phase 3 — Optional social layer with privacy-first design

AI-Powered Features (Coach, Predictions, Suggestions)

- **Reason**: Requires ML infrastructure, training data, and significant R&D investment
- **Future Plans**: Phase 2/3 — Progressive AI integration starting with rule-based recommendations

Third-Party Integrations (Apple Health, Google Fit, Fitbit)

- **Reason**: Requires OAuth implementations, API maintenance, and platform-specific testing
- **Future Plans**: Phase 3 — Integration marketplace approach

Monetization Features (Subscriptions, Premium Tiers)

- **Reason**: Focus on building core value first; monetization adds billing complexity
- **Future Plans**: Phase 2 — Freemium model with premium analytics and AI features

Offline-First Mode with Background Sync

- **Reason**: Adds significant complexity to data management and conflict resolution
- **Future Plans**: Phase 2 — Service worker implementation for basic offline functionality

5.3 Future Enhancements

Phase 2 Candidates:

- **AI Habit Suggestions**: Rule-based then ML-powered habit recommendations based on user goals and patterns
- **Streak Freeze / Vacation Mode**: Protect streaks during planned absences (limited uses per month)
- **Habit Templates Library**: Community-curated templates for common routines (morning routine, fitness, study)
- **Progressive Difficulty**: Habits auto-increase difficulty as user builds consistency
- **Push Notifications**: Browser push and potential PWA notification support

Long-term Vision:

- **AI Personal Coach**: Conversational AI providing daily guidance, accountability, and adaptive recommendations
- **Wellness Score**: Holistic score combining habit completion, consistency, variety, and improvement
- **Social Accountability**: Opt-in buddy system, group challenges, community support
- **Marketplace**: Third-party integrations, custom themes, plugin system
- **Enterprise Version**: Team habits, organization-wide wellness programs

---

6. Functional Requirements

> **Note**: All requirements tagged with MoSCoW priority: **[Must Have]**, **[Should Have]**, **[Could Have]**, **[Won't Have]**

6.1 Core Functional Requirements

FR-001: User Registration [Must Have]

- **Requirement**: The system shall allow new users to register with email address and password, validate inputs, and create a secure account.
- **Business Value**: Account creation is the gateway to all functionality; without it, no user engagement occurs.
- **User Story Reference**: US-01
- **Inputs**: Email (string, valid format), Password (string, ≥8 chars with complexity), Display name (string, 2-50 chars)
- **Processing**: Validate email format and uniqueness; hash password with bcrypt (12 rounds); create user record; generate verification token; send verification email
- **Outputs**: Success response with user ID; verification email sent; redirect to "check your email" page
  Business Rules:
  - **Rule 1**: Email must be unique across all accounts (case-insensitive)
  - **Rule 2**: Password must be ≥8 characters with at least 1 uppercase, 1 lowercase, 1 number, 1 special character
  - **Rule 3**: Display name must be 2-50 characters, alphanumeric with spaces allowed
  - **Rule 4**: Verification email must be sent within 30 seconds of registration

  Acceptance Criteria:
  - [ ] User can successfully register with valid email, password, and display name
  - [ ] System rejects registration with existing email and shows helpful error
  - [ ] System rejects password not meeting complexity requirements with specific feedback
  - [ ] Verification email arrives within 30 seconds
  - [ ] User cannot log in before email verification
  - [ ] Password is stored as bcrypt hash, never in plaintext
  - [ ] Registration form shows real-time validation feedback
- **Priority**: Must Have (P0)
- **Success Criteria**: 95%+ registration success rate for valid inputs; 0 plaintext passwords in database

FR-002: User Authentication (Login) [Must Have]

- **Requirement**: The system shall authenticate users via email/password, issue JWT tokens, and manage sessions securely.
- **Business Value**: Secure authentication protects user data and enables personalized experiences.
- **User Story Reference**: US-02
- **Inputs**: Email (string), Password (string), "Remember Me" flag (boolean)
- **Processing**: Validate credentials against stored hash; generate access token (JWT, 15min expiry) and refresh token (7-day expiry or 30-day if "Remember Me"); set secure HTTP-only cookies; log login event
- **Outputs**: Authenticated session; redirect to dashboard; JWT tokens set in secure cookies

  Business Rules:
  - **Rule 1**: Maximum 5 failed login attempts per email per 15-minute window; after 5 failures, lock for 15 minutes
  - **Rule 2**: Access tokens expire in 15 minutes; refresh tokens in 7 days (or 30 with "Remember Me")
  - **Rule 3**: All login attempts (success/failure) logged with timestamp and IP
  - **Rule 4**: Concurrent sessions allowed (max 5 devices)

  Acceptance Criteria:
  - [ ] User can log in with valid email and password
  - [ ] System rejects invalid credentials with generic "Invalid email or password" message
  - [ ] JWT access token generated with 15-minute expiry
  - [ ] Refresh token enables silent re-authentication before access token expires
  - [ ] Account locks after 5 failed attempts for 15 minutes
  - [ ] "Remember Me" extends session to 30 days
  - [ ] Login works correctly after email verification
  - [ ] Tokens stored in HTTP-only secure cookies (not localStorage)
- **Priority**: Must Have (P0)
- **Success Criteria**: 99.9% authentication success rate for valid credentials; 0 unauthorized access incidents

FR-003: Password Recovery [Must Have]

- **Requirement**: The system shall allow users to reset their password via a secure email-based flow.
- **Business Value**: Prevents permanent account lockout; reduces support burden.
- **User Story Reference**: US-03
- **Inputs**: Email address (for request); reset token + new password (for reset)
- **Processing**: Validate email exists; generate unique reset token (UUID, 1-hour expiry); send email with reset link; on submission, validate token, hash new password, update record, invalidate token
- **Outputs**: Email sent confirmation; password updated confirmation; redirect to login

  Business Rules:
  - **Rule 1**: Reset token expires after 1 hour
  - **Rule 2**: Reset token is single-use (invalidated after successful reset)
  - **Rule 3**: System always shows "If an account exists, we've sent a reset email" (no email enumeration)
  - **Rule 4**: New password must meet same complexity requirements as registration

  Acceptance Criteria:
  - [ ] User receives reset email within 30 seconds when account exists
  - [ ] System shows same message whether email exists or not (prevent enumeration)
  - [ ] Reset link expires after 1 hour
  - [ ] Reset link works only once
  - [ ] New password must meet complexity requirements
  - [ ] After reset, all existing sessions are invalidated
  - [ ] User can log in with new password immediately after reset
- **Priority**: Must Have (P0)
- **Success Criteria**: Password reset success rate > 98%; email delivery within 30 seconds

FR-004: Dashboard Display [Must Have]

- **Requirement**: The system shall display a personalized dashboard showing today's habits, productivity score, streak information, and key metrics upon login.
- **Business Value**: Dashboard provides immediate value and context, driving daily engagement and reducing time-to-action.
- **User Story Reference**: US-04, US-08, US-23
- **Inputs**: Authenticated user session; current date/time; user's habits, completions, streaks
- **Processing**: Fetch today's scheduled habits; calculate productivity score (completed/scheduled × 100); fetch current streak data; load motivational quote; calculate weekly statistics
- **Outputs**: Rendered dashboard with: welcome message, productivity score, today's habits with toggles, streak counter, XP/level display, motivational quote, mini calendar, upcoming reminders, weekly chart

  Business Rules:
  - **Rule 1**: Productivity score = (habits completed today / habits scheduled today) × 100, rounded to integer
  - **Rule 2**: Welcome message is time-aware ("Good morning/afternoon/evening, [Name]")
  - **Rule 3**: Motivational quote rotates daily (not repeated within 30 days)
  - **Rule 4**: Dashboard data refreshes on focus/tab-switch (stale data < 60 seconds)

  Acceptance Criteria:
  - [ ] Dashboard loads within 2 seconds of authentication
  - [ ] Productivity score accurately reflects today's completion ratio
  - [ ] All today's scheduled habits appear with correct completion state
  - [ ] Streak counter shows accurate current streak for each habit
  - [ ] Level and XP bar display correct values
  - [ ] Motivational quote changes daily
  - [ ] Weekly chart shows last 7 days of completion data
  - [ ] Upcoming reminders show next 5 scheduled items
  - [ ] Dashboard is fully responsive (mobile, tablet, desktop)
- **Priority**: Must Have (P0)
- **Success Criteria**: Dashboard load time < 2s (p95); daily active user engagement > 80%

FR-005: Habit Creation [Must Have]

- **Requirement**: The system shall allow users to create habits with configurable name, category, priority, difficulty, schedule, and optional metadata.
- **Business Value**: Habit creation is the primary interaction that drives all subsequent engagement and data generation.
- **User Story Reference**: US-05, US-20, US-21, US-30
- **Inputs**: Habit name (string, 1-100), description (optional, 0-500), category (enum), priority (enum), difficulty (enum), schedule (object: type + days), color (hex), icon (string)
- **Processing**: Validate all inputs; create habit record; set default values for optional fields; calculate first scheduled occurrence; update user's habit count
- **Outputs**: Created habit object with ID; success notification; habit appears in list and calendar

  Business Rules:
  - **Rule 1**: Maximum 50 active habits per user (archived habits don't count)
  - **Rule 2**: Habit name must be 1-100 characters (can duplicate — user's choice)
  - **Rule 3**: Default values: priority=Medium, difficulty=Medium, schedule=Daily, category=General
  - **Rule 4**: Categories: Health, Productivity, Learning, Wellness, Finance, Social, Creative, Custom
  - **Rule 5**: Difficulties: Easy (10 XP), Medium (20 XP), Hard (30 XP), Expert (50 XP)
  - **Rule 6**: Priorities: Low, Medium, High, Critical

  Acceptance Criteria:

    - [ ] User can create habit with minimum required field (name only, defaults applied)
    - [ ] User can configure all optional fields (category, priority, difficulty, schedule, color, icon)
    - [ ] System rejects habit name exceeding 100 characters
    - [ ] System enforces 50-habit limit with clear messaging
    - [ ] Created habit appears on dashboard for its next scheduled day
    - [ ] Created habit appears on calendar on scheduled days
    - [ ] Creation form provides visual feedback for all selections
    - [ ] Custom schedule allows selecting any combination of weekdays
- **Priority**: Must Have (P0)
- **Success Criteria**: Habit creation takes < 30 seconds (median); 90%+ success rate for creation attempts

FR-006: Habit Completion Tracking [Must Have]

- **Requirement**: The system shall allow users to mark habits as complete/incomplete for the current day, with immediate UI feedback, XP awards, and streak updates.
- **Business Value**: Completion tracking is the core value loop — every completion generates data, awards XP, and reinforces behavior.
- **User Story Reference**: US-06, US-07, US-15
- **Inputs**: Habit ID, completion action (complete/uncomplete), optional note (string, 0-500 chars)
- **Processing**: Record completion with timestamp; calculate XP award; update streak (increment on complete, recalculate on uncomplete); update productivity score; check milestone triggers; apply optimistic UI update
- **Outputs**: Updated completion state; XP award notification; updated streak count; updated productivity score; milestone celebration (if triggered)

  Business Rules:
    - **Rule 1**: Users can only complete habits scheduled for the current day
    - **Rule 2**: XP awarded: Easy=10, Medium=20, Hard=30, Expert=50
    - **Rule 3**: Streak increments only if habit was scheduled (weekday habits don't break streak on weekends)
    - **Rule 4**: "Perfect Day" bonus: +25 XP when all scheduled habits are completed
    - **Rule 5**: Undo window: 5 seconds after completion, user can undo without affecting streak
    - **Rule 6**: After undo window, uncompleting deducts XP and may break streak

  Acceptance Criteria:
    - [ ] Completion toggle responds within 100ms (optimistic update)
    - [ ] XP is correctly calculated and awarded per difficulty level
    - [ ] Streak increments on completion and displays correctly
    - [ ] Productivity score updates in real-time after completion
    - [ ] "Perfect Day" bonus triggers when last habit is completed
    - [ ] Undo is available for 5 seconds via toast notification
    - [ ] Optional note can be attached to completion (max 500 chars)
    - [ ] Completion state persists across page refreshes
    - [ ] Cannot complete habits not scheduled for today
- **Priority**: Must Have (P0)
- **Success Criteria**: Completion interaction < 100ms UI response; 0 data inconsistencies between streak count and actual completions

FR-007: Habit Edit & Delete [Must Have]

- **Requirement**: The system shall allow users to edit habit configurations and soft-delete or archive habits.
- **Business Value**: Users' needs evolve; editing and cleanup capabilities ensure the habit list stays relevant and manageable.
- **User Story Reference**: US-09, US-10

- **Inputs**: Habit ID + updated fields (edit); Habit ID + action type: delete/archive (removal)
- **Processing**: Edit: validate updated fields, update record, recalculate schedule if changed. Delete: soft-delete (set `deleted_at` timestamp, retain 30 days). Archive: set status to "archived", remove from active views.
- **Outputs**: Updated habit object; confirmation notification; UI reflects changes immediately
  Business Rules:
  - **Rule 1**: Edit takes effect immediately for future scheduled instances
  - **Rule 2**: Editing schedule does not retroactively change completion history
  - **Rule 3**: Soft-delete retains data for 30 days (recoverable)
  - **Rule 4**: Archive preserves all history but removes from active lists and dashboard
  - **Rule 5**: Permanent deletion occurs 30 days after soft-delete (automated cleanup)

  Acceptance Criteria:
  - [ ] All habit fields are editable (name, category, priority, difficulty, schedule, color, icon)
  - [ ] Edit saves successfully and reflects immediately in all views
  - [ ] Schedule changes apply to future occurrences only
  - [ ] Delete shows confirmation dialog with "Archive instead?" option
  - [ ] Deleted habits are recoverable within 30 days
  - [ ] Archived habits retain statistics and completion history
  - [ ] Archived habits don't appear in dashboard or active habit lists
  - [ ] Users can unarchive habits, restoring them to active status
- **Priority**: Must Have (P0)
- **Success Criteria**: Edit operations complete in < 500ms; 0 data loss from accidental deletion (30-day recovery)

FR-008: Calendar Views [Must Have]

- **Requirement**: The system shall provide daily, weekly, and monthly calendar views showing habit schedules and completion history.
- **Business Value**: Calendar visualization helps users understand their habit patterns temporally and maintain schedule awareness.
- **User Story Reference**: US-11, US-12
- **Inputs**: Selected date/range, view type (daily/weekly/monthly), optional category filter
- **Processing**: Fetch habits scheduled in date range; fetch completion records for range; render appropriate calendar view with completion indicators
- **Outputs**: Rendered calendar view with habit cards/indicators showing completion status per day
  Business Rules:
  - **Rule 1**: Daily view shows full detail (all habits for that day with times and completion status)
  - **Rule 2**: Weekly view shows summary (habit count and completion ratio per day)
  - **Rule 3**: Monthly view shows color-coded intensity (green gradient based on completion %)
  - **Rule 4**: Default view opens to current week; monthly view highlights today
  - **Rule 5**: Navigation supports keyboard arrows and swipe gestures

  Acceptance Criteria:
  - [ ] Calendar loads within 1.5 seconds for any view type
  - [ ] Daily view shows all habits for selected day with accurate completion states
  - [ ] Weekly view shows 7 days with per-day completion indicators
  - [ ] Monthly view shows full month with color-coded day cells
  - [ ] User can navigate between days/weeks/months
  - [ ] "Today" button jumps to current date from any position
  - [ ] Calendar accurately reflects habit schedule changes
  - [ ] Calendar is responsive and usable on mobile (scrollable day view)
- **Priority**: Must Have (P0)

- **Success Criteria**: Calendar renders correctly for all views; navigation feels instant (< 300ms transitions)

FR-009: Analytics Dashboard [Must Have]

- **Requirement**: The system shall display analytics including completion rates, trends, category breakdowns, and time-series charts across configurable time ranges.
- **Business Value**: Analytics transform raw tracking data into actionable insights, enabling users to optimize their routines.
- **User Story Reference**: US-14, US-22, US-27, US-29
- **Inputs**: Time range (daily/weekly/monthly/yearly), optional category filter, optional habit filter
- **Processing**: Aggregate completion data for selected range; calculate rates, trends, comparisons; render charts and metric cards
- **Outputs**: Analytics dashboard with: overall completion rate, per-category bar chart, trend indicators, line chart (completion over time), best/worst habits, streak statistics

  Business Rules:
  - **Rule 1**: Minimum 3 days of data required to show trend indicators
  - **Rule 2**: Minimum 7 days required for meaningful weekly analytics
  - **Rule 3**: Trends calculated as: current period completion % vs. previous period completion %
  - **Rule 4**: "Improving" if ≥5% increase; "Declining" if ≥5% decrease; "Stable" otherwise
  - **Rule 5**: Data aggregation uses user's configured timezone

  Acceptance Criteria:
  - [ ] Analytics page loads within 2 seconds with up to 365 days of data
  - [ ] Overall completion rate displays as percentage with trend arrow
  - [ ] Category breakdown shows per-category completion rates as bar/pie chart
  - [ ] Line chart shows completion trend over selected time range
  - [ ] Trend indicators show "Improving", "Declining", or "Stable" per habit
  - [ ] Time range selector works correctly (daily/weekly/monthly/yearly)
  - [ ] Category filter updates all charts simultaneously
  - [ ] Empty state shown gracefully when insufficient data exists
  - [ ] Charts are accessible (screen reader descriptions, keyboard navigable)
- **Priority**: Must Have (P0)
- **Success Criteria**: Analytics accurately represent underlying data (verified via unit tests); page load < 2s

FR-010: XP & Leveling System [Must Have]

- **Requirement**: The system shall award XP for habit completions and streaks, calculate user level, and display progress toward next level.
- **Business Value**: XP and leveling provide tangible progress markers that drive long-term engagement through the core motivation loop.
- **User Story Reference**: US-15, US-18
- **Inputs**: Completion events, streak milestones, special achievements
- **Processing**: Calculate XP per event (difficulty-based + bonuses); update cumulative XP; calculate level (floor(sqrt(totalXP / 100))); determine progress to next level
- **Outputs**: Updated XP total, current level, XP to next level progress bar, level-up celebration (when triggered)

  Business Rules:
  - **Rule 1**: XP per completion: Easy=10, Medium=20, Hard=30, Expert=50
  - **Rule 2**: Streak bonuses: 7-day=+50 XP, 14-day=+100 XP, 30-day=+200 XP, 60-day=+350 XP, 100-day=+500 XP, 365-day=+2000 XP
  - **Rule 3**: Perfect Day bonus: +25 XP
  - **Rule 4**: Level formula: Level = floor(sqrt(totalXP / 100))

- **Rule 5**: XP is never negative; uncompleting deducts minimum of (habit XP, available XP this level)
- **Rule 6**: Level-down is possible if significant XP deducted (rare edge case)

Acceptance Criteria:

- [ ] XP awarded correctly for each difficulty level
- [ ] Streak bonuses trigger at exact milestone days
- [ ] Level calculation is accurate per formula
- [ ] XP progress bar shows percentage to next level
- [ ] Level-up triggers celebration animation
- [ ] Total XP displayed on dashboard and profile
- [ ] XP history is auditable (per-event log)
- [ ] Perfect Day bonus triggers when all today's habits are completed

- **Priority**: Must Have (P0)
- **Success Criteria**: XP calculations are 100% accurate (verified by unit tests); level-up animation plays within 500ms of threshold

FR-011: Reminder System [Should Have]

- **Requirement**: The system shall allow users to configure reminders for habits and display upcoming reminders on the dashboard with optional browser notifications.
- **Business Value**: Reminders prevent habit forgetting, the #1 reason for broken streaks, directly improving completion rates.
- **User Story Reference**: US-13, US-26
- **Inputs**: Habit ID, reminder time, reminder frequency, notification preferences
- **Processing**: Store reminder configuration; schedule notifications based on user timezone; display upcoming reminders on dashboard; trigger browser notifications at scheduled times
- **Outputs**: Reminder cards on dashboard; browser notifications at configured times; "upcoming" section showing next 5 reminders

  Business Rules:

  - **Rule 1**: One reminder per habit maximum
  - **Rule 2**: Reminders respect user's notification preferences (can be disabled globally)
  - **Rule 3**: Quiet hours apply (default 10pm-7am, configurable)
  - **Rule 4**: Browser notifications require explicit user permission
  - **Rule 5**: Dashboard shows next 5 upcoming reminders sorted by time

  Acceptance Criteria:

  - [ ] Users can set reminder time for each habit
  - [ ] Dashboard shows upcoming reminders section with next 5 items
  - [ ] Browser notifications fire at configured time (when permitted)
  - [ ] Quiet hours suppress notifications in configured window
  - [ ] User can disable all reminders globally from settings
  - [ ] Reminders adjust for timezone changes
  - [ ] Reminder for completed habit is suppressed/marked done
- **Priority**: Should Have (P1)
- **Success Criteria**: Reminders fire within ±1 minute of scheduled time; 15%+ improvement in completion rate for habits with reminders

FR-012: Badge & Achievement System [Should Have]

- **Requirement**: The system shall award badges for reaching defined milestones and display them in an achievement gallery.
- **Business Value**: Badges provide collection-driven motivation and milestone recognition that deepen engagement.

- **User Story Reference**: US-17, US-39
- **Inputs**: Completion events, streak updates, XP milestones, special conditions
- **Processing**: After each relevant event, check all badge conditions; award newly qualified badges; store award timestamp; trigger celebration
- **Outputs**: Badge award notification; updated achievement gallery; badge count increment

  Business Rules:
  - **Rule 1**: Badges are permanently earned (never revoked, even if streak breaks later)
  - **Rule 2**: Badge check runs after each completion and streak update
  - **Rule 3**: Multiple badges can be earned simultaneously
  - **Rule 4**: Badge rarity: Common (>50% users earn), Rare (10-50%), Epic (1-10%), Legendary (<1%)
  - **Rule 5**: Badge gallery shows earned badges prominently, locked badges as silhouettes with requirements

  Acceptance Criteria:
  - [ ] Badges awarded correctly when conditions are met
  - [ ] Badge notification appears with animation and description
  - [ ] Achievement gallery shows all available badges (earned + locked)
  - [ ] Locked badges show requirements to unlock
  - [ ] Badge rarity displayed for each badge
  - [ ] Badges are never revoked once earned
  - [ ] Multiple simultaneous badges are queued and shown sequentially
  - [ ] Badge count displayed on profile/dashboard
- **Priority**: Should Have (P1)
- **Success Criteria**: All badge conditions trigger accurately; badge gallery renders correctly with 50+ badge types

FR-013: Goal Management [Should Have]

- **Requirement**: The system shall allow users to create, track, and manage goals with progress visualization linked to habit performance.
- **Business Value**: Goals provide direction and purpose for habits, connecting daily actions to meaningful outcomes.
- **User Story Reference**: US-16
- **Inputs**: Goal title (string), description (string), type (daily/weekly/monthly/custom), target metric, linked habits, deadline
- **Processing**: Create goal record; calculate progress based on linked habit completions; update progress on each completion; determine goal status (active/achieved/failed)
- **Outputs**: Goal cards with progress bars; goal status updates; goal achievement celebrations

  Business Rules:
  - **Rule 1**: Goals can be linked to 1+ habits (progress = linked habits' average completion %)
  - **Rule 2**: Daily goals reset at midnight (user timezone)
  - **Rule 3**: Goal is "Achieved" when target metric is met within deadline
  - **Rule 4**: Goal is "Failed" when deadline passes without meeting target (can be reactivated)
  - **Rule 5**: Maximum 20 active goals per user

  Acceptance Criteria:
  - [ ] Users can create goals with title, type, target, and deadline
  - [ ] Goals can be linked to one or more habits
  - [ ] Progress bar updates in real-time as linked habits are completed
  - [ ] Daily goals reset at midnight
  - [ ] Goal achievement triggers celebration notification
  - [ ] Failed goals show "Missed" status with option to restart
  - [ ] Goal history shows past goals with achievement/failure status

    - [ ] Maximum 20 active goals enforced with clear messaging
- **Priority**: Should Have (P1)
- **Success Criteria**: Goal progress accuracy 100% (unit tested); goal creation < 20 seconds

FR-014: Settings & Preferences [Must Have]

- **Requirement**: The system shall provide a settings interface for theme, notifications, account management, and data operations.
- **Business Value**: Customization improves user satisfaction and accessibility; data management ensures compliance and trust.
- **User Story Reference**: US-25, US-24
- **Inputs**: Setting key-value pairs (theme, notification prefs, language, timezone, etc.)
- **Processing**: Validate setting values; persist to user preferences; apply changes immediately (theme) or on next relevant event (notifications)
- **Outputs**: Updated user experience reflecting new settings; confirmation of changes

  Business Rules:
    - **Rule 1**: Theme change applies immediately without page reload
    - **Rule 2**: Data export includes all user data in selected format (GDPR compliance)
    - **Rule 3**: Account deletion requires password confirmation and has 30-day grace period
    - **Rule 4**: Settings persist across sessions and devices

  Acceptance Criteria:
    - [ ] Dark/Light/System theme toggle works immediately without reload
    - [ ] Notification preferences save and apply correctly
    - [ ] Timezone selection affects all date/time displays
    - [ ] Data export generates complete file in < 30 seconds
    - [ ] Account deletion requires password and has 30-day recovery window
    - [ ] All settings persist across browser sessions
    - [ ] Settings page is accessible and screen-reader friendly
- **Priority**: Must Have (P0)
- **Success Criteria**: Theme switch < 100ms; data export contains 100% of user data

6.2 User Interface Requirements

UI-001: Global Layout & Navigation

- **Description**: Responsive application shell with sidebar navigation, header, and main content area

  User Interactions:
    - Sidebar: Navigate between Dashboard, Habits, Calendar, Analytics, Goals, Achievements, Reminders, Settings
    - Header: View user avatar, XP/level, quick actions, notifications bell
    - Mobile: Collapsible sidebar becomes bottom navigation bar

  Layout Requirements:
    - Sidebar: 240px width on desktop, collapsible to icons (64px), hidden on mobile
    - Content area: Fluid width with max-width 1400px, centered
    - Header: Fixed top, 64px height, contains breadcrumbs and user menu
- **Responsive Behavior**: Desktop (≥1024px): full sidebar + content; Tablet (768-1023px): collapsed sidebar + content; Mobile (<768px): bottom tab bar + full-width content
- **Accessibility Requirements**: All navigation items keyboard-accessible; aria-labels on icon-only buttons; focus indicators visible; skip-to-content link

- **Visual Design Notes**: Soft shadows on sidebar; smooth slide transition for sidebar collapse; active route highlighted with accent color

UI-002: Dashboard Page

- **Description**: Central hub showing today's overview with habit cards, metrics, and widgets
  User Interactions:
  - Click habit toggle to complete/uncomplete
  - Click habit card to navigate to habit detail
  - Click calendar preview to navigate to full calendar
  - Click "See All" links to navigate to respective modules
  Layout Requirements:
  - Grid layout: 2-column on desktop (left: habits & calendar, right: stats & reminders)
  - Single column on mobile with stacked cards
  - Cards with 16px padding, 12px border-radius, subtle shadow
- **Responsive Behavior**: Grid columns collapse to single column below 768px; cards stack vertically; stats cards become horizontal scroll on mobile
- **Accessibility Requirements**: Habit toggles have aria-checked state; productivity score has aria-valuenow; cards have semantic headings

UI-003: Habit Creation/Edit Modal

- **Description**: Modal/slide-over form for creating and editing habits
  User Interactions:
  - Text input for name and description
  - Dropdown/select for category, priority, difficulty
  - Toggle buttons for schedule days
  - Color picker for habit color
  - Icon selector grid
  - Submit/Cancel buttons
  Layout Requirements:
  - Modal: 480px max-width, centered on desktop; full-screen slide-up on mobile
  - Form fields stacked vertically with 16px spacing
  - Action buttons fixed at bottom
- **Responsive Behavior**: Desktop: centered modal with backdrop. Mobile: full-screen slide-up panel
- **Accessibility Requirements**: Focus trapped within modal; Escape key closes; form labels associated with inputs; error messages announced to screen readers

UI-004: Analytics Charts

- **Description**: Data visualization components displaying habit analytics
  User Interactions:
  - Hover on data points for tooltips
  - Click legend items to toggle data series
  - Select time range via segmented control
  - Filter by category via dropdown
  Layout Requirements:
  - Charts fill container width with responsive height
  - Minimum chart height: 200px on mobile, 300px on desktop
  - Legend positioned below chart on mobile, beside chart on desktop
- **Responsive Behavior**: Charts resize fluidly; axis labels reduce on small screens; touch-friendly tooltip triggers

- **Accessibility Requirements**: Charts have `role="img"` with aria-label describing the data; tabular data fallback available; high contrast mode compatible

6.3 Data Requirements

DR-001: Users

Description: User account information and authentication data

Data Fields:

Field Name	Data Type	Required	Validation Rules	Notes
`id`	UUID	Yes	Auto-generated	Primary key
`email`	VARCHAR(255)	Yes	Valid email format, unique	Login identifier
`password_hash`	VARCHAR(255)	Yes	bcrypt hash	Never expose via API
`display_name`	VARCHAR(50)	Yes	2-50 chars, alphanumeric	Shown in UI
`avatar_url`	TEXT	No	Valid URL or null	Profile image
`timezone`	VARCHAR(50)	Yes	Valid IANA timezone	Default: UTC
`theme`	ENUM	Yes	'light', 'dark', 'system'	Default: system
`email_verified`	BOOLEAN	Yes	—	Default: false
`created_at`	TIMESTAMPTZ	Yes	Auto-set	Account creation
`updated_at`	TIMESTAMPTZ	Yes	Auto-updated	Last modification
`deleted_at`	TIMESTAMPTZ	No	—	Soft delete marker

Data Relationships: One-to-many with habits, completions, goals, achievements

Data Volume: Expected 10,000 users in first year

Data Retention: Active accounts: indefinite. Deleted accounts: purged 30 days after deletion

Data Privacy: Email is PII; password_hash never exposed; GDPR export includes all user data

DR-002: Habits

Description: Habit definitions and configuration

Data Fields:

Field Name	Data Type	Required	Validation Rules	Notes
`id`	UUID	Yes	Auto-generated	Primary key
`user_id`	UUID	Yes	FK → users.id	Owner
`name`	VARCHAR(100)	Yes	1-100 chars	Habit name
`description`	TEXT	No	0-500 chars	Optional description
`category`	ENUM	Yes	Valid category	Health, Productivity, etc.
`priority`	ENUM	Yes	Low/Medium/High/Critical	Default: Medium
`difficulty`	ENUM	Yes	Easy/Medium/Hard/Expert	Determines XP
`schedule_type`	ENUM	Yes	daily/weekdays/weekends/custom	Repeat pattern
`schedule_days`	INTEGER[]	No	0-6 (Sun-Sat)	For custom schedule
`time_of_day`	ENUM	No	morning/afternoon/evening/anytime	Ordering preference
`color`	VARCHAR(7)	No	Valid hex color	Default: category color
`icon`	VARCHAR(50)	No	Valid icon name	Default: category icon
`status`	ENUM	Yes	active/archived/deleted	Default: active
`streak_current`	INTEGER	Yes	≥ 0	Default: 0
`streak_longest`	INTEGER	Yes	≥ 0	Default: 0
`created_at`	TIMESTAMPTZ	Yes	Auto-set	—
`updated_at`	TIMESTAMPTZ	Yes	Auto-updated	—
`archived_at`	TIMESTAMPTZ	No	—	Archive timestamp
`deleted_at`	TIMESTAMPTZ	No	—	Soft delete

Data Relationships: Belongs to user; has many completions; linked to goals (many-to-many)

Data Volume: Average 12 habits per user; ~120,000 habits at 10K users

Data Retention: Active: indefinite. Deleted: 30 days post-delete. Archived: indefinite

Data Privacy: User-scoped (RLS); never exposed to other users

DR-003: Habit Completions

Description: Record of each habit completion event

Data Fields:

Field Name	Data Type	Required	Validation Rules	Notes
`id`	UUID	Yes	Auto-generated	Primary key
`habit_id`	UUID	Yes	FK → habits.id	Which habit
`user_id`	UUID	Yes	FK → users.id	Redundant for RLS performance
`completed_date`	DATE	Yes	Valid date	The day completed
`completed_at`	TIMESTAMPTZ	Yes	Auto-set	Exact timestamp
`xp_earned`	INTEGER	Yes	≥ 0	XP awarded
`note`	TEXT	No	0-500 chars	Completion note
`is_bonus`	BOOLEAN	No	—	If part of bonus (streak, perfect day)

Data Relationships: Belongs to habit and user; unique constraint on (habit_id, completed_date)

Data Volume: Average 8 completions/user/day; ~29M records/year at 10K users

Data Retention: Indefinite (core analytics data)

Data Privacy: User-scoped; included in data export

DR-004: Achievements (Badges)

Description: Badge definitions and user award records

Data Fields:

Field Name	Data Type	Required	Validation Rules	Notes
`id`	UUID	Yes	Auto-generated	Badge type ID
`name`	VARCHAR(100)	Yes	Unique	Badge name
`description`	TEXT	Yes	—	How to earn
`icon`	VARCHAR(50)	Yes	Valid icon	Badge icon
`rarity`	ENUM	Yes	common/rare/epic/legendary	Rarity level
`category`	ENUM	Yes	streak/volume/category/special	Badge type
`condition`	JSONB	Yes	Valid condition object	Unlock criteria

User Achievements Junction:

Field Name	Data Type	Required	Validation Rules	Notes
`user_id`	UUID	Yes	FK → users.id	Who earned it
`badge_id`	UUID	Yes	FK → achievements.id	Which badge
`earned_at`	TIMESTAMPTZ	Yes	Auto-set	When earned
`xp_awarded`	INTEGER	Yes	≥ 0	Bonus XP for badge

Data Relationships: Many-to-many between users and badges

Data Volume: ~50 badge types; average 15 badges per active user

Data Retention: Indefinite (never revoked)

Data Privacy: User-scoped; visible only to badge owner

DR-005: Goals

Description: User-defined goals linked to habits

Data Fields:

| Field Name | Data Type | Required | Validation Rules | Notes | |---|---|---|---|---| | `id` | UUID | Yes | Auto-generated | Primary key | | `user_id` | UUID | Yes | FK → users.id | Owner | | `title` | VARCHAR(200) | Yes | 1-200 chars | Goal title | | `description` | TEXT | No | 0-500 chars | Goal description | | `type` | ENUM | Yes | daily/weekly/monthly/custom | Goal timeframe | | `target_value` | INTEGER | Yes | > 0 | Target percentage or count | | `current_value` | INTEGER | Yes | ≥ 0 | Current progress | | `deadline` | DATE | No | Future date or null | Goal deadline | | `status` | ENUM | Yes | active/achieved/failed/paused | Goal status | | `created_at` | TIMESTAMPTZ | Yes | Auto-set | — | | `achieved_at` | TIMESTAMPTZ | No | — | When achieved |

Data Relationships: Belongs to user; linked to habits (many-to-many via goal_habits junction)

Data Volume: Average 5 goals per user

Data Retention: Indefinite (historical record)

Data Privacy: User-scoped

DR-006: Reminders

Description: User-configured habit reminders

Data Fields:

| Field Name | Data Type | Required | Validation Rules | Notes | |---|---|---|---|---| | `id` | UUID | Yes | Auto-generated | Primary key | | `user_id` | UUID | Yes | FK → users.id | Owner | | `habit_id` | UUID | Yes | FK → habits.id | Linked habit | | `reminder_time` | TIME | Yes | Valid time | When to remind | | `is_active` | BOOLEAN | Yes | — | Default: true | | `last_triggered` | TIMESTAMPTZ | No | — | Last fire time |

Data Relationships: Belongs to user and habit; one reminder per habit max

Data Volume: ~60% of habits have reminders; ~72,000 reminders at scale

Data Retention: Deleted when habit is permanently deleted

Data Privacy: User-scoped

6.4 Integration Requirements

INT-001: Supabase Authentication

- **Description**: Integration with Supabase Auth for user authentication and session management
- **Integration Type**: SDK / REST API
- **Data Flow**: Bidirectional — client sends credentials, receives tokens; server validates tokens on each request
- **Data Exchanged**: Credentials (login), JWT tokens (session), user metadata (profile)
- **Frequency**: Every authenticated request (token validation); login/register (auth events)
- **Authentication**: Supabase anon key for public endpoints; service_role key for server-side admin operations
- **Error Handling**: Token expired → silent refresh via refresh token; refresh failed → redirect to login; auth service down → show maintenance page
- **Fallback Strategy**: Retry with exponential backoff (3 attempts); graceful degradation showing cached data if available

INT-002: Supabase Database (PostgreSQL)

- **Description**: PostgreSQL database via Supabase for all application data storage

- **Integration Type**: Supabase Client SDK (real-time subscriptions + REST)
- **Data Flow**: Bidirectional — CRUD operations from client and server; real-time subscriptions for live updates
- **Data Exchanged**: All domain entities (habits, completions, goals, achievements, reminders)
- **Frequency**: High — multiple reads/writes per user session (average 20-50 operations per active session)
- **Authentication**: Row Level Security (RLS) policies enforcing user-scoped data access
- **Error Handling**: Constraint violations → user-friendly error messages; connection loss → queue operations, retry on reconnect; query timeout → pagination/optimization
- **Fallback Strategy**: Optimistic UI updates with rollback on failure; retry failed writes 3x with exponential backoff

INT-003: Supabase Storage (Optional)

- **Description**: File storage for user avatars and potential future media
- **Integration Type**: Supabase Storage SDK
- **Data Flow**: Upload (client → storage); retrieval (storage → client via CDN URL)
- **Data Exchanged**: Image files (avatars, max 2MB, JPEG/PNG/WebP)
- **Frequency**: Low — avatar upload/change (rare event per user)
- **Authentication**: Authenticated uploads only; public read via signed URLs
- **Error Handling**: Upload failure → show error, retain previous avatar; oversized file → client-side validation with clear message
- **Fallback Strategy**: Default avatar (generated initials) if upload fails or no avatar set

INT-004: Browser Notifications API

- **Description**: Web Notifications API for habit reminder notifications
- **Integration Type**: Browser API (client-side)
- **Data Flow**: Application → Browser → User (one-way notifications)
- **Data Exchanged**: Notification title, body, icon, action URL
- **Frequency**: Per-reminder schedule (0-10 per user per day)
- **Authentication**: Requires user permission grant (Notification.requestPermission)
- **Error Handling**: Permission denied → fallback to in-app notifications only; service worker unavailable → in-app reminders only
- **Fallback Strategy**: In-app notification badges and dashboard reminder section always available regardless of browser notification permission

6.5 Business Rules

BR-001: Streak Calculation

- **Rule**: A habit streak increments by 1 for each consecutive scheduled day the habit is completed. The streak breaks (resets to 0) if a scheduled day is missed.
- **Conditions**: Habit is in "active" status; the day is a scheduled day for that habit
- **Actions**: On completion: streak_current += 1; if streak_current > streak_longest then streak_longest = streak_current. On missed: streak_current = 0.
- **Exceptions**: Weekday-only habits don't break streak on weekends; holidays/vacation mode (P2) preserves streak
- **Example**: Habit scheduled Mon-Fri. User completes Mon-Thu, misses Fri → streak breaks. Sat-Sun don't affect it. Monday completion starts new streak at 1.

BR-002: XP Economy

- **Rule**: XP is the primary currency earned through habit completions and milestones. XP determines user level and unlocks achievements.

- **Conditions**: User completes a scheduled habit; user reaches a streak milestone; user achieves "Perfect Day"
- **Actions**: Award XP based on source (difficulty for completions, fixed amounts for milestones); update total XP; recalculate level
- **Exceptions**: Undo within 5s doesn't deduct XP; admin adjustments possible for bug corrections
- **Example**: User completes "Hard" meditation → +30 XP. Same user's 7th consecutive day → +50 XP streak bonus. All 5 habits done → +25 XP Perfect Day. Total earned this session: 105 XP.

BR-003: Productivity Score

- **Rule**: Daily productivity score = (habits completed today / habits scheduled today) × 100, rounded to nearest integer.
- **Conditions**: At least 1 habit is scheduled for today
- **Actions**: Score recalculates in real-time after each completion/uncompletion
- **Exceptions**: If 0 habits scheduled (e.g., habit-free day), score displays as "—" (not 0 or 100)
- **Example**: 3 habits scheduled today, 2 completed → score = 67%. Third completed → score = 100%.

BR-004: Active Habit Limit

- **Rule**: Users may have a maximum of 50 active (non-archived, non-deleted) habits at any time.
- **Conditions**: User attempts to create a new habit when at 50 active habits
- **Actions**: Block creation; display message: "You've reached the 50-habit limit. Archive or delete inactive habits to create new ones."
- **Exceptions**: Archived and deleted habits don't count toward limit
- **Example**: User with 49 active habits creates one more → allowed (now 50). Attempts 51st → blocked with helpful message.

BR-005: Data Isolation

- **Rule**: Users can only access their own data. No user can view, modify, or infer another user's habits, completions, or statistics.
- **Conditions**: Every database query and API request
- **Actions**: Supabase RLS policies enforce user_id = auth.uid() on all tables; API endpoints validate ownership
- **Exceptions**: System-level analytics (aggregate, anonymized) for product improvement
- **Example**: User A queries habits endpoint → receives only User A's habits. Attempting to access User B's habit ID → 404 (not 403, to prevent ID enumeration).

BR-006: Badge Permanence

- **Rule**: Once a badge is earned, it is permanently awarded and can never be revoked regardless of subsequent activity changes.
- **Conditions**: Badge condition met at any point in time
- **Actions**: Badge permanently stored in user_achievements table
- **Exceptions**: None — even if streak breaks after earning "30-Day Streak" badge, badge remains
- **Example**: User earns "7-Day Streak" badge on day 7. On day 8, user uncompletes day 7 habit (breaking streak). Badge remains earned.

---

7. Non-Functional Requirements (NFRs)

7.1 Performance Requirements

NFR-P-001: Response Time

- **Requirement**: Application pages and interactions must meet specified response time targets under normal load conditions.
  Details:
  - **Page Load Time (Initial)**: < 2.5 seconds for first contentful paint (FCP); < 3.5 seconds for time-to-interactive (TTI)
  - **Page Navigation (Subsequent)**: < 500ms for client-side route transitions
  - **API Response Time**: < 300ms for 95th percentile; < 1s for 99th percentile
  - **UI Interaction Response**: < 100ms for toggle/button interactions (optimistic updates)
  - **Database Query Time**: < 200ms for standard queries; < 1s for complex aggregations (analytics)
- **Measurement Method**: Lighthouse CI in pipeline; Supabase query monitoring; client-side performance API measurements
- **Test Conditions**: Normal load (100 concurrent users); standard network conditions (4G equivalent)

NFR-P-002: Throughput

- **Requirement**: System must support specified concurrent user load without degradation.
  Details:
  - **Concurrent Users**: Support 1,000 concurrent users with no performance degradation
  - **Peak Load**: Handle 5,000 concurrent users with ≤20% response time increase
  - **Transactions Per Second**: Handle 500 habit completion events per second
  - **Database Connections**: Pool up to 100 concurrent Supabase connections
- **Measurement Method**: Load testing with k6 or Artillery; Supabase dashboard metrics

NFR-P-003: Scalability

- **Requirement**: Architecture must support growth without fundamental redesign.
  Details:
  - **Horizontal Scaling**: Next.js deployed on Vercel with automatic edge function scaling
  - **Database Scaling**: Supabase Pro plan with connection pooling (PgBouncer); read replicas for analytics queries
  - **Data Volume**: Support 50M+ completion records without query degradation (proper indexing)
  - **User Growth**: Architecture supports 100,000 users without structural changes

7.2 Security Requirements

NFR-S-001: Authentication Security

- **Requirement**: Authentication system must resist common attack vectors and protect user credentials.
  Details:
  - **Method**: Supabase Auth (built on GoTrue) with JWT tokens
  - **Token Storage**: HTTP-only secure cookies (not localStorage)
  - **Session Duration**: Access token: 15 minutes; Refresh token: 7 days (30 with "Remember Me")
  - **Password Policy**: Minimum 8 characters, 1 uppercase, 1 lowercase, 1 number, 1 special character
  - **Brute Force Protection**: Rate limit: 5 attempts per 15 minutes per email; exponential backoff
  - **Password Storage**: bcrypt with 12 salt rounds

NFR-S-002: Authorization & Access Control

- **Requirement**: All data access must be properly authorized with user-scoped isolation.
  Details:
  - **Access Control Model**: Row Level Security (RLS) on all Supabase tables
  - **Policy**: `user_id = auth.uid()` on SELECT, INSERT, UPDATE, DELETE for all user-owned tables

- **API Protection**: All endpoints require valid JWT; ownership verified server-side
- **Data Isolation**: Users can never access another user's data; 404 returned for unauthorized resource access (not 403)

NFR-S-003: Data Protection

- **Requirement**: All data must be protected in transit and at rest.
  Details:
  - **Encryption in Transit**: TLS 1.3 for all communications; HSTS headers enforced
  - **Encryption at Rest**: Supabase encrypts all data at rest (AES-256)
  - **PII Handling**: Email classified as PII; available for GDPR export/deletion; never logged in plaintext
  - **Data Masking**: Passwords never returned in API responses; email partially masked in UI where appropriate
  - **Input Sanitization**: All user inputs sanitized to prevent XSS; parameterized queries prevent SQL injection

NFR-S-004: Compliance

- **Requirement**: Application must meet relevant data protection standards.
  Standards:
  - **GDPR**: Right to access (data export), right to erasure (account deletion), right to portability (JSON/CSV export), consent for notifications
  - **OWASP Top 10**: Protection against injection, broken auth, sensitive data exposure, XSS, CSRF, security misconfiguration
  - **Content Security Policy**: Strict CSP headers preventing inline script execution and unauthorized resource loading

7.3 Reliability & Availability

NFR-R-001: Uptime

- **Requirement**: Application must maintain high availability for user access.
  Details:
  - **SLA**: 99.5% uptime (allows ~43 hours downtime per year)
  - **Planned Maintenance**: Zero-downtime deployments via Vercel (blue-green deployment)
  - **Dependency Uptime**: Supabase guarantees 99.9% uptime on Pro plan

NFR-R-002: Fault Tolerance

- **Requirement**: System must gracefully handle component failures.
  Details:
  - **Optimistic Updates**: UI continues to function during brief network interruptions
  - **Retry Logic**: Failed API calls retry 3x with exponential backoff (1s, 2s, 4s)
  - **Graceful Degradation**: If analytics service is slow, dashboard shows cached data with "Last updated" indicator
  - **Error Boundaries**: React error boundaries prevent full-page crashes from component errors

NFR-R-003: Disaster Recovery

- **Requirement**: Data must be recoverable in case of catastrophic failure.
  Details:
  - **RTO (Recovery Time Objective)**: < 4 hours for full service restoration
  - **RPO (Recovery Point Objective)**: < 1 hour of data loss maximum
  - **Backup Strategy**: Supabase daily automatic backups; point-in-time recovery on Pro plan

- **Recovery Procedures**: Restore from Supabase backup; redeploy from Git main branch

7.4 Usability & Accessibility

NFR-U-001: User Experience

- **Requirement**: Application must be intuitive, efficient, and delightful to use.
  Details:
  - **Intuitiveness**: New users can create and complete their first habit within 3 minutes without documentation
  - **Consistency**: All interactive elements follow consistent patterns (buttons, forms, cards, modals)
  - **Feedback**: Every user action produces visible feedback within 100ms (loading states, success/error toasts)
  - **Error Messages**: All errors are user-friendly, specific, and actionable (never show raw error codes)
  - **Animations**: Framer Motion animations enhance understanding (not distract); respect `prefers-reduced-motion`

NFR-U-002: Accessibility

- **Requirement**: Application must meet WCAG 2.1 Level AA compliance.
  Details:
  - **WCAG Compliance**: Level AA for all pages and components
  - **Screen Reader Support**: All content accessible via screen readers (NVDA, VoiceOver, JAWS)
  - **Keyboard Navigation**: Full functionality achievable via keyboard alone; logical tab order
  - **Color Contrast**: Minimum 4.5:1 contrast ratio for normal text; 3:1 for large text
  - **Alternative Text**: All images have descriptive alt text; decorative images have empty alt
  - **Focus Indicators**: Visible focus rings on all interactive elements (not removed for aesthetics)
  - **Motion Sensitivity**: Respect `prefers-reduced-motion` media query; provide option to disable animations

NFR-U-003: Internationalization

- **Requirement**: Application architecture must support future multi-language deployment.
  Details:
  - **Languages Supported (V1)**: English (US) only
  - **Architecture**: All user-facing strings externalized in locale files (i18n-ready)
  - **Locale Support**: Dates, times, numbers formatted per user timezone setting
  - **Future RTL Support**: Layout system compatible with RTL languages (CSS logical properties)

7.5 Compatibility

NFR-C-001: Browser Compatibility

- **Requirement**: Application must function correctly on modern browsers.
  Platforms:
  - **Chrome**: Version 90+ (Windows, macOS, Linux, Android)
  - **Firefox**: Version 88+ (Windows, macOS, Linux)
  - **Safari**: Version 14+ (macOS, iOS)
  - **Edge**: Version 90+ (Windows)
  - **Mobile Browsers**: Chrome Mobile, Safari Mobile (iOS 14+, Android 10+)

NFR-C-002: Device Compatibility

- **Requirement**: Application must be fully functional across device sizes.
  Devices:

- **Desktop**: 1024px+ width; full feature set with optimized layouts
- **Tablet**: 768-1023px; adapted layouts with collapsible navigation
- **Mobile**: 320-767px; mobile-optimized layouts with bottom navigation, touch-friendly targets (min 44px)

7.6 Maintainability

NFR-M-001: Code Quality

- **Requirement**: Codebase must maintain high quality standards for long-term maintainability.
  Details:
  - **Code Coverage**: > 80% for business logic; > 60% overall
  - **Type Safety**: Full TypeScript strict mode; no any types in production code
  - **Code Review**: All changes via PR with at least 1 review
  - **Documentation**: JSDoc comments on all exported functions; README per module
  - **Linting**: ESLint + Prettier enforced via pre-commit hooks; zero warnings in CI

NFR-M-002: Monitoring & Observability

- **Requirement**: Application must provide visibility into performance and errors.
  Details:
  - **Error Tracking**: Sentry or similar for runtime error capture with source maps
  - **Performance Monitoring**: Web Vitals tracking (LCP, FID, CLS) via Next.js Analytics or custom
  - **Logging**: Structured logging for server-side operations; log levels (debug, info, warn, error)
  - **Alerts**: Critical error rate spike alerts; performance degradation alerts
  - **Analytics**: User engagement metrics (DAU, session duration, feature usage)

---

8. Implementation Decisions

8.1 Architectural Decisions

AD-001: Full-Stack Framework Selection

- **Decision**: Use Next.js (App Router) as the full-stack framework
- **Context**: Need SSR for SEO, API routes for backend logic, and modern React features
  Options Considered:
  - **Option 1**: Next.js App Router — Full-stack, SSR/SSG, API routes, React Server Components
  - **Option 2**: Remix — Full-stack, nested routing, form-based mutations
  - **Option 3**: Create React App + Express — Separate frontend/backend, more flexibility
- **Selected Option**: Next.js App Router — industry standard, Vercel deployment optimization, RSC for performance, large ecosystem
- **Consequences**: Tied to Vercel ecosystem for optimal deployment; must use App Router patterns; server/client component boundary management
- **Trade-offs**: Gain: SSR/SSG, edge functions, automatic optimization. Sacrifice: Some flexibility in deployment targets

AD-002: Database & Backend Platform

- **Decision**: Use Supabase as the backend-as-a-service platform
- **Context**: Need PostgreSQL database, authentication, real-time capabilities, and storage without managing infrastructure

Options Considered:
  - **Option 1**: Supabase — PostgreSQL, Auth, Storage, Real-time, Edge Functions
  - **Option 2**: Firebase — NoSQL, Auth, Real-time, serverless functions
  - **Option 3**: Custom PostgreSQL + custom auth — Full control, more maintenance
- **Selected Option**: Supabase — PostgreSQL's relational power with BaaS convenience; open-source; SQL-based; excellent DX
- **Consequences**: Dependent on Supabase service; RLS as primary security model; must design within Supabase's patterns
- **Trade-offs**: Gain: Rapid development, managed infrastructure, built-in auth. Sacrifice: Some backend customization flexibility

AD-003: State Management Approach

- **Decision**: Use React Context + custom hooks for state management (no external library)
- **Context**: Need lightweight, maintainable state management without over-engineering
  Options Considered:
  - **Option 1**: React Context + Hooks — Built-in, no dependencies, sufficient for this scale
  - **Option 2**: Zustand — Lightweight, minimal boilerplate
  - **Option 3**: Redux Toolkit — Industry standard, time-travel debugging, but heavy for this use case
- **Selected Option**: React Context + Hooks — zero additional dependencies; sufficient for user-scoped state; combined with server state via SWR/React Query patterns
- **Consequences**: Context re-render considerations for deeply nested state; may need optimization with useMemo/useCallback
- **Trade-offs**: Gain: Zero dependencies, simple mental model. Sacrifice: No built-in devtools, manual optimization needed for large state trees

AD-004: Animation Library

- **Decision**: Use Framer Motion for all animations and micro-interactions
- **Context**: Premium UX requires smooth, performant animations that enhance usability
  Options Considered:
  - **Option 1**: Framer Motion — Declarative, React-native, powerful gesture support
  - **Option 2**: CSS Animations + Tailwind — Zero JS overhead, limited interactivity
  - **Option 3**: React Spring — Physics-based, smaller bundle
- **Selected Option**: Framer Motion — best-in-class React animation library; layout animations; gesture support; AnimatePresence for exit animations
- **Consequences**: Adds ~30KB to bundle (tree-shakeable); must be careful with animation count for performance
- **Trade-offs**: Gain: Professional animations with minimal code; layout animations. Sacrifice: Bundle size increase; performance consideration for many simultaneous animations

8.2 Technology Stack Decisions

Component	Technology	Version	Rationale
Frontend Framework	Next.js (App Router)	14.x	SSR, API routes, RSC, Vercel optimization

UI Styling	Tailwind CSS	3.x	Utility-first, rapid development, consistent design system
Animation	Framer Motion	10.x	Premium animations, layout transitions, gesture support
Icons	React Icons (+ Lucide)	4.x	Comprehensive icon sets, tree-shakeable
Database	Supabase (PostgreSQL)	Latest	Managed PostgreSQL, RLS, real-time subscriptions
Authentication	Supabase Auth	Latest	JWT, social auth, email verification built-in
State Management	React Context + Hooks	—	Lightweight, no additional dependencies
Charts	Recharts or Chart.js (React wrapper)	Latest	Responsive charts, accessibility support
Form Handling	React Hook Form + Zod	Latest	Performance, validation, type safety
Date Handling	date-fns	Latest	Lightweight, tree-shakeable, immutable
Deployment	Vercel	—	Next.js optimized, edge network, CI/CD
Type Safety	TypeScript	5.x	Strict mode, full type coverage

8.3 Modules & Components

Module 1: Authentication Module (`/app/(auth)`)

- **Purpose**: Handle user registration, login, password recovery, and session management
- **Responsibilities**: Credential validation; token management; protected route enforcement; profile management
- **Interfaces**: AuthContext providing user state, login/logout/register functions to all components

- **Dependencies**: Supabase Auth; form validation (Zod); secure cookie management

Module 2: Dashboard Module (`/app/(dashboard)/dashboard`)

- **Purpose**: Provide personalized daily overview with actionable habit information
- **Responsibilities**: Aggregate today's data; display habits with completion toggles; show statistics cards; render mini-charts
- **Interfaces**: Consumes HabitContext and AnalyticsContext; emits completion events
- **Dependencies**: Habits module; Analytics module; Gamification module; Calendar module

Module 3: Habits Module (`/app/(dashboard)/habits`)

- **Purpose**: Full habit lifecycle management (CRUD) and daily tracking
- **Responsibilities**: Habit creation/editing/deletion/archival; completion tracking; streak management; scheduling
- **Interfaces**: HabitContext providing habits list, CRUD operations, completion functions
- **Dependencies**: Supabase database; Gamification module (XP events); Reminder module (linked reminders)

Module 4: Calendar Module (`/app/(dashboard)/calendar`)

- **Purpose**: Temporal visualization of habit schedules and completion history
- **Responsibilities**: Render daily/weekly/monthly views; display heatmap; show habit history per date
- **Interfaces**: Consumes habit data and completions; provides date selection to other modules
- **Dependencies**: Habits module (schedule data); Completions data; date-fns

Module 5: Analytics Module (`/app/(dashboard)/analytics`)

- **Purpose**: Transform tracking data into visual insights and trends
- **Responsibilities**: Data aggregation; chart rendering; trend calculation; export generation
- **Interfaces**: Consumes all completion and habit data; provides analytics API
- **Dependencies**: Habits module; Chart library (Recharts); data export utilities

Module 6: Gamification Module (`/app/(dashboard)/achievements`)

- **Purpose**: Manage XP, levels, badges, and milestone celebrations
- **Responsibilities**: XP calculation and award; level progression; badge condition evaluation; celebration triggers
- **Interfaces**: GamificationContext providing XP, level, badges; event listeners for completions
- **Dependencies**: Habits module (completion events); Supabase (achievement storage)

Module 7: Goals Module (`/app/(dashboard)/goals`)

- **Purpose**: Goal setting, tracking, and progress visualization
- **Responsibilities**: Goal CRUD; progress calculation; goal-habit linking; status management
- **Interfaces**: GoalContext providing goals list, CRUD operations, progress data
- **Dependencies**: Habits module (linked habits); Analytics module (progress data)

Module 8: Reminders Module (`/app/(dashboard)/reminders`)

- **Purpose**: Configure and deliver habit reminders
- **Responsibilities**: Reminder configuration; scheduling; browser notification delivery; dashboard display
- **Interfaces**: ReminderContext providing reminder list, create/edit operations
- **Dependencies**: Habits module (linked habits); Browser Notifications API; service worker (future)

Module 9: Settings Module (`/app/(dashboard)/settings`)

- **Purpose**: Application customization and user account management
- **Responsibilities**: Theme management; notification preferences; data export/import; account operations
- **Interfaces**: SettingsContext providing theme, preferences; export/import functions
- **Dependencies**: All modules (theme applies globally); Supabase (data operations)

8.4 API Contracts

API Endpoint 1: Habits CRUD

- **Method & Path**: `GET /api/habits` | `POST /api/habits` | `PUT /api/habits/[id]` | `DELETE /api/habits/[id]`
- **Purpose**: Full CRUD operations for user habits
- **Request Format**: POST/PUT: `{ name, description?, category, priority, difficulty, schedule, color?, icon? }`
- **Response Format**: `{ success: boolean, data: Habit | Habit[], error?: string }`
- **Error Handling**: 400 (validation), 401 (unauthenticated), 404 (not found/not owned), 429 (rate limited)

API Endpoint 2: Completions

- **Method & Path**: `POST /api/completions` | `DELETE /api/completions/[id]`
- **Purpose**: Record and remove habit completions
- **Request Format**: POST: `{ habit_id, date, note? }` | DELETE: path param ID
- **Response Format**: `{ success: boolean, data: { completion, xp_earned, streak_update, badges_earned? }, error?: string }`
- **Error Handling**: 400 (invalid date/habit), 401 (unauthenticated), 409 (already completed today)

API Endpoint 3: Analytics

- **Method & Path**: `GET /api/analytics?range=weekly&category=health`
- **Purpose**: Fetch aggregated analytics data for charts and insights
- **Request Format**: Query params: `range` (daily|weekly|monthly|yearly), `category?`, `habit_id?`, `start_date?`, `end_date?`
- **Response Format**: `{ completion_rate, streak_data, category_breakdown, trend_data, heatmap_data }`
- **Error Handling**: 400 (invalid range), 401 (unauthenticated), 404 (no data for range)

API Endpoint 4: Goals

- **Method & Path**: `GET /api/goals` | `POST /api/goals` | `PUT /api/goals/[id]` | `DELETE /api/goals/[id]`
- **Purpose**: Goal management operations
- **Request Format**: POST/PUT: `{ title, description?, type, target_value, deadline?, linked_habit_ids? }`
- **Response Format**: `{ success: boolean, data: Goal | Goal[], error?: string }`
- **Error Handling**: 400 (validation), 401 (unauthenticated), 404 (not found)

API Endpoint 5: User Profile & Settings

- **Method & Path**: `GET /api/user/profile` | `PUT /api/user/profile` | `PUT /api/user/settings`
- **Purpose**: Profile and settings management
- **Request Format**: PUT profile: `{ display_name?, avatar_url?, timezone? }` | PUT settings: `{ theme, notifications, language }`
- **Response Format**: `{ success: boolean, data: UserProfile | UserSettings }`

- **Error Handling**: 400 (validation), 401 (unauthenticated)

API Endpoint 6: Data Export

- **Method & Path**: `GET /api/user/export?format=csv`
- **Purpose**: GDPR-compliant full data export
- **Request Format**: Query param: `format` (csv|json)
- **Response Format**: File download (Content-Disposition: attachment)
- **Error Handling**: 401 (unauthenticated), 503 (export service busy — retry later)

8.5 Database Schema Decisions

Entity 1: users

- **Purpose**: Store user accounts and preferences
- **Key Fields**: id, email, password_hash, display_name, avatar_url, timezone, theme, xp_total, level
- **Relationships**: Has many habits, completions, goals, user_achievements, reminders
- **Indexes**: Unique index on email; index on created_at

Entity 2: habits

- **Purpose**: Store habit definitions and current state
- **Key Fields**: id, user_id, name, category, priority, difficulty, schedule_type, schedule_days, status, streak_current, streak_longest
- **Relationships**: Belongs to user; has many completions; many-to-many with goals
- **Indexes**: Index on (user_id, status); index on (user_id, category); index on (user_id, schedule_type)

Entity 3: habit_completions

- **Purpose**: Record each completion event (source of truth for analytics)
- **Key Fields**: id, habit_id, user_id, completed_date, completed_at, xp_earned, note
- **Relationships**: Belongs to habit and user
- **Indexes**: Unique index on (habit_id, completed_date); index on (user_id, completed_date); index on completed_date for range queries

Entity 4: achievements (badge_definitions)

- **Purpose**: Define available badges and their unlock conditions
- **Key Fields**: id, name, description, icon, rarity, category, condition (JSONB)
- **Relationships**: Many-to-many with users via user_achievements
- **Indexes**: Index on category; index on rarity

Entity 5: user_achievements

- **Purpose**: Track which badges each user has earned
- **Key Fields**: user_id, badge_id, earned_at, xp_awarded
- **Relationships**: Junction table between users and achievements
- **Indexes**: Composite primary key (user_id, badge_id); index on earned_at

Entity 6: goals

- **Purpose**: User-defined goals with progress tracking
- **Key Fields**: id, user_id, title, type, target_value, current_value, status, deadline
- **Relationships**: Belongs to user; many-to-many with habits via goal_habits

- **Indexes**: Index on (user_id, status); index on deadline

Entity 7: reminders

- **Purpose**: Habit reminder configurations
- **Key Fields**: id, user_id, habit_id, reminder_time, is_active
- **Relationships**: Belongs to user and habit
- **Indexes**: Index on (user_id, is_active); index on reminder_time

---

9. Testing Strategy & Decisions

9.1 Testing Philosophy

What Makes a Good Test:

- Tests external behavior and user outcomes, not implementation details
- Tests are independent and can run in any order
- Tests are fast (< 5s for unit, < 30s for integration) and reliable (zero flaky tests)
- Test names follow pattern: "should [expected behavior] when [condition]"
- Use meaningful assertions with clear failure messages
- Test the happy path AND edge cases/error scenarios

9.2 Testing Levels

9.2.1 Unit Testing

- **Scope**: Individual functions, hooks, utility modules, business logic calculations
- **Coverage Target**: > 80% for business logic (XP calculation, streak logic, productivity score); > 60% overall
  Modules to Test:
  - XP calculation engine: all difficulty levels, bonuses, edge cases
  - Streak calculation: consecutive days, skipped non-scheduled days, reset scenarios
  - Productivity score: various completion ratios, edge cases (0 habits, all complete)
  - Badge condition evaluation: each badge type's unlock logic
  - Form validation: all input validation rules
  - Date utilities: timezone handling, schedule calculations
- **Prior Art**: Jest + React Testing Library; Vitest for faster execution

9.2.2 Integration Testing

- **Scope**: API routes, database operations, authentication flows, module interactions
- **Coverage Target**: All critical API endpoints tested with success/failure scenarios
  Modules to Test:
  - Auth flow: register → verify → login → refresh → logout
  - Habit lifecycle: create → complete → edit → archive → delete
  - Completion + XP: complete habit → verify XP awarded → verify streak updated
  - Goal progress: complete linked habit → verify goal progress updates
  - Data export: request export → verify all user data included
- **Prior Art**: Supertest for API testing; Supabase test utilities

9.2.3 End-to-End Testing

- **Scope**: Complete user workflows through the browser
- **Coverage Target**: All critical user journeys (onboarding, daily check-in, analytics review)
  User Journeys to Test:
  - Journey 1: New user registration → first habit creation → first completion → badge earned
  - Journey 2: Returning user login → view dashboard → complete multiple habits → check updated score
  - Journey 3: Analytics navigation → time range change → category filter → data export
  - Journey 4: Settings change → theme toggle → verify immediate visual update
  - Journey 5: Habit management → edit habit → verify calendar update → archive habit
- **Prior Art**: Playwright or Cypress for E2E; testing against Supabase test instance

9.2.4 Performance Testing

- **Scope**: Load times, concurrent user handling, database query performance
  Test Scenarios:
  - **Load Testing**: 100 concurrent users performing normal operations (habit CRUD, completions)
  - **Stress Testing**: 1,000 concurrent users; verify graceful degradation
  - **Database Performance**: Analytics queries with 1M+ completion records; verify < 1s response
  - **Bundle Size**: Verify JavaScript bundle < 200KB gzipped for initial load
- **Tools**: Lighthouse CI; k6 for load testing; Supabase query analyzer

9.2.5 Security Testing

- **Scope**: Authentication bypass, data leakage, injection attacks, authorization failures
  Test Types:
  - **RLS Verification**: Attempt to access other users' data via direct Supabase queries
  - **Auth Bypass**: Test protected routes without valid tokens; test expired tokens
  - **Input Injection**: XSS payloads in habit names/notes; SQL injection attempts
  - **Rate Limiting**: Verify brute force protection on login
  - **CSRF**: Verify state-changing operations require valid CSRF tokens

9.3 Test Seams

Seam 1: Supabase Client Layer

- **Location**: Between React components and Supabase SDK calls
- **Type**: Unit + Integration
- **Rationale**: Mock Supabase client for unit tests; use test instance for integration tests

Seam 2: API Route Layer

- **Location**: Next.js API routes (`/app/api/`)
- **Type**: Integration
- **Rationale**: Test request/response contracts independent of UI; verify auth middleware

Seam 3: Business Logic Layer

- **Location**: Utility functions for XP, streaks, badges, scoring
- **Type**: Unit
- **Rationale**: Pure functions with no dependencies; highest coverage priority

Seam 4: UI Component Layer

- **Location**: React components with user interaction

- **Type**: Unit (RTL) + E2E
- **Rationale**: Test rendering and interaction behavior; verify accessibility

9.4 Test Data Strategy

- **Test Data Source**: Seed scripts generating realistic habit/completion data; factory functions for test fixtures
- **Data Refresh Strategy**: Each test suite uses fresh data; database reset between integration test suites
- **PII Handling**: Test data uses generated fake emails (faker.js); no real user data in test environments

---

10. Assumptions & Constraints

10.1 Assumptions

> ■■ **CRITICAL**: These assumptions must be validated before development begins

Assumption 1: Supabase free tier is sufficient for development and initial launch (up to 500MB database, 2GB bandwidth)

- **Impact if Wrong**: May need to upgrade to Pro ($25/mo) earlier than planned, increasing initial costs
- **Validation Method**: Estimate data volume for first 1,000 users; verify against Supabase free tier limits

Assumption 2: Users have modern browsers with JavaScript enabled and accept browser notifications

- **Impact if Wrong**: Notification-based reminders won't work; need fallback reminder strategy (email)
- **Validation Method**: Analytics on target audience browser distribution; implement graceful degradation

Assumption 3: Vercel free tier supports expected traffic for initial launch (100GB bandwidth, serverless function limits)

- **Impact if Wrong**: Need to upgrade Vercel plan or explore alternative hosting
- **Validation Method**: Estimate page weights × expected monthly visits; compare to Vercel limits

Assumption 4: Users are willing to create accounts (no anonymous/guest mode needed for MVP)

- **Impact if Wrong**: Onboarding friction may reduce trial users; may need guest mode in Phase 2
- **Validation Method**: Monitor registration-to-completion conversion rate; A/B test if needed

Assumption 5: English-only interface is acceptable for initial launch

- **Impact if Wrong**: May lose non-English speaking users; accelerate i18n implementation
- **Validation Method**: Analyze traffic sources for language preferences post-launch

10.2 Constraints

10.2.1 Technical Constraints

Constraint 1: Must use Next.js with App Router (specified tech stack)

- **Reason**: Portfolio demonstration requirement; alignment with modern React practices
- **Workaround**: N/A — this is a deliberate choice, not a limitation

Constraint 2: Must use Supabase for database and auth (no custom backend infrastructure)

- **Reason**: Reduce infrastructure management burden; faster development velocity

- **Workaround**: Supabase Edge Functions for custom server logic if needed

Constraint 3: No native mobile apps in V1 (web-responsive only)

- **Reason**: Scope control; web-first development prioritizes broader access
- **Workaround**: Responsive design ensures mobile usability; PWA consideration for Phase 2

10.2.2 Budget Constraints

- **Total Budget**: $0-$100/month (free tiers + minimal paid services)
  Budget Allocation:
  - **Supabase**: Free tier ($0) → Pro ($25/mo) when needed
  - **Vercel**: Free tier ($0) → Pro ($20/mo) if traffic exceeds limits
  - **Domain**: ~$12/year for custom domain
  - **Error Tracking (Sentry)**: Free tier ($0) for 5K events/month
  - **Contingency**: $50/month buffer for unexpected service needs

10.2.3 Timeline Constraints

- **Project Duration**: 16 weeks (4 months)
- **Hard Deadlines**: None (portfolio project, flexible timeline)
  Milestones:
  - Week 1-2: PRD, Architecture, Project Setup
  - Week 3-4: Auth + Dashboard foundation
  - Week 5-8: Core modules (Habits, Calendar, Analytics)
  - Week 9-12: Secondary modules (Reminders, Achievements, Goals, Settings)
  - Week 13-14: Integration testing, polish, performance optimization
  - Week 15-16: E2E testing, deployment, documentation

10.2.4 Resource Constraints

- **Team Size**: 1 full-stack developer (solo project)
- **Skill Availability**: Full-stack proficiency in Next.js, React, TypeScript, Supabase, Tailwind CSS
- **Third-party Dependencies**: Limited to free/open-source tools; no paid API dependencies for core features

10.2.5 Business Constraints

- **Open Source Consideration**: Code must be clean enough for potential open-sourcing
- **Portfolio Quality**: Every component must demonstrate professional-grade engineering

---

11. Dependencies & Risks

11.1 Dependencies

ID	Dependency	Description	Owner	Status	Due Date	Impact if Delayed	Mitigation

DEP-001	Supabase Project Setup	Supabase project creation, database schema deployment, RLS policies	Developer	Pending	Week 1	Blocks all data operations	Early setup; local Supabase for development
DEP-002	Vercel Deployment	Vercel project configuration, environment variables, CI/CD pipeline	Developer	Pending	Week 2	Blocks deployment; manual deploys as workaround	Set up in Week 1 alongside Supabase
DEP-003	Design System & Components	Tailwind config, color palette, base components (Button, Card, Input, Modal)	Developer	Pending	Week 2	Blocks all UI development	Build incrementally; start with essentials
DEP-004	Authentication System	Supabase Auth configuration, login/register flows, session management	Developer	Pending	Week 3	Blocks all authenticated features	Priority implementation; use Supabase UI library as scaffold
DEP-005	Chart Library Selection	Final selection and configuration of charting library (Recharts vs Chart.js)	Developer	Pending	Week 5	Blocks analytics module	Evaluate both in Week 4; default to Recharts

DEP-006	Badge Definitions	Complete list of badges with conditions, icons, and descriptions	Developer	Pending	Week 9	Blocks achievements module	Define minimum set (10 badges) first, expand later

11.2 Risks & Mitigation Strategies

Risk ID	Risk Description	Impact	Probability	Risk Score	Mitigation Strategy	Contingency Plan	Owner
RISK-001	Supabase free tier limits exceeded during development	Medium	Low	3	Monitor usage; optimize queries; use connection pooling	Upgrade to Pro plan ($25/mo)	Developer
RISK-002	Performance issues with large analytics datasets	High	Medium	6	Proper indexing; pagination; server-side aggregation; caching	Database function-based aggregation; limit date ranges	Developer
RISK-003	Complex streak calculation bugs	Medium	Medium	4	Comprehensive unit tests for streak logic; property-based testing	Streak recalculation cron job as safety net	Developer
RISK-004	Scope creep from AI features temptation	Medium	High	6	Strict phase boundaries; AI features explicitly deferred to Phase 2	Document AI readiness hooks but don't implement	Developer

RISK-005	Solo developer burnout over 16-week timeline	High	Medium	6	Incremental milestones with deployable states; celebrate progress	Extend timeline if needed; reduce P2 scope	Developer
RISK-006	Browser notification API inconsistency	Low	Medium	3	Graceful degradation; in-app reminders as primary; notifications as enhancement	In-app-only reminders with visual cues	Developer
RISK-007	Tailwind + Framer Motion performance at scale	Medium	Low	3	Performance budget enforcement; lazy loading; animation count limits	Reduce animations; optimize Tailwind purge	Developer
RISK-008	Data loss from accidental deletion or Supabase outage	High	Low	4	Soft deletes everywhere; Supabase auto-backups; 30-day recovery	Point-in-time recovery on Pro plan	Developer

---

12. Timeline & Milestones

12.1 Project Phases

Phase	Milestone	Duration	Start Date	Target Date	Status	Dependencies
Planning	PRD + Architecture Complete	2 weeks	Week 1	Week 2	In Progress	None

Foundation	Auth + Dashboard + Design System	2 weeks	Week 3	Week 4	Pending	DEP-001, DEP-002
Core Development	Habits + Calendar + Analytics	4 weeks	Week 5	Week 8	Pending	DEP-003, DEP-004
Secondary Development	Reminders + Achievements + Goals + Settings	4 weeks	Week 9	Week 12	Pending	Core modules
Polish & Testing	Integration testing + Performance + Accessibility	2 weeks	Week 13	Week 14	Pending	All modules
Launch	Deployment + Documentation + Monitoring	2 weeks	Week 15	Week 16	Pending	All testing

12.2 Detailed Milestone Breakdown

Phase 1: Planning & Architecture (Weeks 1-2)

- **Week 1**: PRD creation; Supabase project setup; database schema design; Vercel project configuration
- **Week 2**: Architecture document; coding specs; task breakdown; design system planning; component library evaluation

Phase 2: Foundation (Weeks 3-4)

- **Week 3**: Next.js scaffolding; Tailwind configuration; base component library (Button, Card, Input, Modal, Sidebar); Authentication module (register, login, password reset)
- **Week 4**: Dashboard layout; protected routes; profile page; theme system (dark/light); responsive navigation

Phase 3: Core Development (Weeks 5-8)

- **Week 5-6**: Habits module (CRUD, completion tracking, streak logic, scheduling); XP system
- **Week 7**: Calendar module (daily/weekly/monthly views, heatmap)
- **Week 8**: Analytics module (charts, trends, category breakdowns, data aggregation)

Phase 4: Secondary Development (Weeks 9-12)

- **Week 9**: Achievements module (badge system, level display, gallery)

- **Week 10**: Goals module (goal CRUD, progress tracking, habit linking)
- **Week 11**: Reminders module (configuration, browser notifications, dashboard cards)
- **Week 12**: Settings module (preferences, data export/import, account management)

Phase 5: Polish & Testing (Weeks 13-14)

- **Week 13**: Integration testing; E2E tests for critical journeys; performance optimization; accessibility audit
- **Week 14**: Bug fixes; UI polish; animation refinement; responsive testing across devices; SEO optimization

Phase 6: Launch (Weeks 15-16)

- **Week 15**: Production deployment; monitoring setup (Sentry, analytics); performance verification
- **Week 16**: Documentation (README, contributing guide); final testing; product launch

---

13. Budget & Resources

13.1 Budget Breakdown

C	It	Cost	Notes
Infrastructure	Supabase (Free → Pro)	$0-$25/mo	Free tier for development; Pro if needed at scale
	Vercel Hosting	$0-$20/mo	Free tier sufficient for initial launch
	Custom Domain	$12/year	Optional; Vercel subdomain available free
Development Tools	GitHub	$0 (Free)	Public/private repos
	VS Code + Extensions	$0	Free IDE
	Figma (Free tier)	$0	Design mockups and prototyping
Third-Party Services	Sentry (Error Tracking)	$0	Free tier: 5K events/month

		Resend (Email Service) $0	Free tier: 100 emails/day (password reset, verification)
	Testing	Playwright/Cypress $0	Open-source testing frameworks
	Contingency	Buffer 5% $50/mo	Unexpected service upgrades
	TOTAL (Monthly)	**$0-$95/mo**	Primarily free-tier services
	TOTAL (Annual)	**$0-$1,152/year**	Scales with user growth

13.2 Resource Allocation

Role	FTE	Duration	Availability	Notes
Full-Stack Developer	1.0	16 weeks	Full-time	Solo developer — all roles
Product Owner	0.1	16 weeks	As needed	Vanga Rudiveek — direction and decisions
Design (self-directed)	0.2	Weeks 1-4	Shared with development	Design system + Figma mockups
QA (self-directed)	0.3	Weeks 13-14	Focused testing phase	Automated + manual testing

---

14. Open Questions

#	Question	Owner	Target Resolution Date	Status	Resolution

1	Should we use Supabase Auth or custom JWT implementation with bcrypt?	Developer	Week 2	Open	Recommendation: Supabase Auth (simpler, battle-tested, includes email verification)
2	Which charting library best balances bundle size, accessibility, and visual quality?	Developer	Week 4	Open	Candidates: Recharts (lightweight, React-native), Chart.js (feature-rich, larger bundle)
3	Should habit completion support backfilling (marking past days as complete)?	Product Owner	Week 5	Open	Consider for P1 with restrictions (max 3 days back)
4	What is the maximum data retention period for free users vs. future premium users?	Product Owner	Week 3	Open	Suggestion: Unlimited for all (storage is cheap)
5	Should the XP/leveling system be visible to other users (leaderboard potential)?	Product Owner	Week 9	Open	V1: Private only. Future: opt-in public leaderboard

6	What email service to use for verification and password reset emails?	Developer	Week 2	Open	Candidates: Resend (modern), Supabase built-in (simpler), SendGrid (free tier)
7	Should we implement PWA capabilities (offline support, install prompt) in V1?	Developer	Week 12	Open	Suggestion: Basic PWA manifest in V1; full offline in V2

---

15. Domain-Specific Considerations

15.1 Gamification & Behavioral Psychology

> **Purpose**: Document the gamification design principles that underpin user engagement

Core Gamification Principles:

- **Variable Reward Schedule**: Mix predictable rewards (XP per completion) with surprise rewards (random badge unlocks, milestone celebrations)
- **Progress Visualization**: Always show users how far they've come (XP bar, streaks, badges collected) — progress is the primary motivator
- **Loss Aversion**: Streaks create "not wanting to lose" motivation — but implement compassionately (streak recovery, gentle messaging on break)
- **Mastery Curve**: Difficulty levels and progressive challenges provide appropriate challenge as users advance
- **Social Proof (Future)**: "Top 20% of users" messaging validates effort without direct competition

XP Economy Design:

- **Currency**: XP (experience points) — single, simple currency
- **Earning Methods**: Habit completion (primary), streak milestones (secondary), Perfect Day bonus, badge bonuses
- **Spending**: XP is not spent — it's cumulative, driving levels. Future: streak freeze costs (P2)
- **Inflation Control**: Fixed XP rates; level formula (sqrt) ensures increasing effort per level

Badge Design Principles:

- **Progressive**: Easy early badges (First Habit, Day 1) building to legendary (365-Day Streak)
- **Varied Categories**: Streak badges, volume badges, category mastery, special event badges
- **Rarity System**: Common (>50% earn) → Rare (10-50%) → Epic (1-10%) → Legendary (<1%)
- **Permanent**: Once earned, never revoked — builds trust and reduces anxiety

15.2 Habit Formation Science

Key Principles Applied:

- **Cue** → **Routine** → **Reward**: Reminders (cue), habit tracking (routine), XP and celebrations (reward)
- **Minimum Viable Habit**: Low-difficulty habits allow building consistency before increasing challenge
- **Habit Stacking**: Routine grouping feature allows users to link habits in sequences
- **21/66 Day Milestones**: Special celebrations at 21 days (common myth) and 66 days (research-backed average for habit formation)
- **Identity-Based Habits**: Level titles reinforce identity ("Habit Champion", "Wellness Master")

15.3 Data Visualization Best Practices

Chart Selection Guidelines:

- **Trends over time**: Line charts (completion rate over days/weeks/months)
- **Comparisons**: Bar charts (category comparison, daily distribution)
- **Composition**: Pie/Donut charts (time-of-day distribution, category breakdown)
- **Density/Patterns**: Heatmaps (GitHub-style contribution map for yearly overview)
- **Progress**: Radial/circular progress bars (goal completion, daily score)

Accessibility in Charts:

- All charts must have text alternative (data table view)
- Color choices distinguishable for color-blind users (colorblind-safe palettes)
- Patterns/textures supplement color coding where possible
- Interactive charts keyboard-navigable with screen reader announcements

---

16. Approval & Sign-off

16.1 Approval Requirements

Role	Name		S	D	Status
Product Owner	Vanga Rudiveek				Pending
Technical Lead	Vanga Rudiveek				Pending
Design Review	Self-review				Pending

16.2 Approval Criteria

This PRD is considered approved when:

- [ ] All sections have been reviewed for completeness and accuracy
- [ ] Technical stack decisions are validated
- [ ] Scope (in/out) is agreed upon
- [ ] Priority assignments (P0/P1/P2) are confirmed

- [ ] Gamification rules (XP, levels, badges) are finalized
- [ ] Data model supports all specified features
- [ ] Non-functional requirements are realistic and measurable
- [ ] Timeline is achievable for solo developer

Appendices

Appendix A: Glossary

Term	Definition
XP	Experience Points — the primary gamification currency earned through habit completions
Streak	Consecutive scheduled days a habit has been completed without missing
Productivity Score	Daily percentage calculated as completed habits / scheduled habits × 100
Heatmap	Visual representation of activity intensity over time (GitHub-style contribution graph)
RLS	Row Level Security — Supabase/PostgreSQL feature ensuring users can only access their own data
Perfect Day	Achieving 100% completion of all scheduled habits for a given day
Badge	Achievement award earned by meeting specific milestones (permanently retained)
Streak Freeze	P2 feature allowing users to preserve streaks during planned absences
MoSCoW	Prioritization method: Must Have, Should Have, Could Have, Won't Have
Optimistic Update	UI pattern where interface updates immediately before server confirmation

TTI	Time to Interactive — performance metric measuring when page becomes fully interactive
RSC	React Server Components — Next.js feature for server-rendered React components

Appendix B: References

- **Nir Eyal - Hooked**: Hook Model framework (Trigger → Action → Variable Reward → Investment)
- **James Clear - Atomic Habits**: Habit loop science (Cue → Craving → Response → Reward)
- **BJ Fogg - Tiny Habits**: Minimum viable habit design; anchor habits
- **Duolingo Gamification Study**: XP, streaks, leagues as engagement drivers
- **WCAG 2.1 Guidelines**: https://www.w3.org/WAI/WCAG21/quickref/
- **Supabase Documentation**: https://supabase.com/docs
- **Next.js App Router Documentation**: https://nextjs.org/docs/app

Appendix C: Related Documents

- **Architecture Document**: `.agents/docs/architecture.md` (to be generated)
- **Coding Specifications**: `.agents/specs/` (to be generated)
- **Governance Rules**: `.agents/rules/` (to be generated)
- **Flow Diagrams**: `.agents/docs/flow-diagrams.md` (to be generated)
- **Test Plan**: `.agents/docs/test-plan.md` (to be generated)